# Enhancing Self-Supervised Video Representations with HAF-STD Net: A Comparative Evaluation Against VLM for Action Recognition




**A Thesis Submitted for the Partial Fulfillment of the Requirements for the Master's in Information Technology**

**Submitted to the**
**Central Department of Computer Science and Information Technology**
**Institute of Science and Technology**
**Tribhuvan University**
**NEPAL**

By

**Erize Rai**
**Exam Roll No. 8025006**
**T.U. Registration No. 5-3-28-193-2023**

**OCTOBER, 2026**

# Declaration

This thesis entitled **"Enhancing Self-Supervised Video Representations with HAF-STD Net: A Comparative Evaluation Against VLM for Action Recognition"** which is being submitted to the Central Department of Computer Science and Information Technology, Institute of Science and Technology (IoST), Tribhuvan University, Nepal for the partial fulfillment of the requirement to the Master of Science Degree in Information Technology has been carried out by me under the supervision of **Asst. Prof. Dr. Dhiraj Kedar Pandey** at Central Department of Computer Science and Information Technology. This work is original and has not been submitted earlier in part or full in this or any other form to any other institution for the award of any degree.

---

**Erize Rai**

Date: OCTOBER, 2026

# Recommendation

This is to recommend that Erize Rai has carried out the thesis entitled **"Enhancing Self-Supervised Video Representations with HAF-STD Net: A Comparative Evaluation Against VLM for Action Recognition"** for the partial fulfillment of the requirement of Master's in Information Technology under my supervision. To the best of my knowledge, this work has not been submitted to any other institution for any academic degree. He has fulfilled all the requirements of the Central Department of Computer Science and Information Technology, Institute of Science and Technology (IoST), Tribhuvan University, Nepal for the submission of the thesis for the Master's degree in Information Technology.

---

**Supervisor**
Asst. Prof. Dr. Dhiraj Kedar Pandey
Assistant Professor
Central Department of Computer Science and
Information Technology
Tribhuvan University
Kathmandu, Nepal

**OCTOBER, 2026**

# Letter of Approval

The thesis work entitled **"Enhancing Self-Supervised Video Representations with HAF-STD Net: A Comparative Evaluation Against VLM for Action Recognition"** submitted at the Central Department of Computer Science and Information Technology, Institute of Science and Technology, Tribhuvan University by Erize Rai has been accepted for the partial fulfillment of Master's in Information Technology.

---

**External Examiner**

Name:
Designation:
Department:
Kathmandu, Nepal

---

**Internal Examiner**

Name:
Designation:
Central Department of Computer Science
and Information Technology
Tribhuvan University
Kathmandu, Nepal

---

**Supervisor**

Asst. Prof. Dr. Dhiraj Kedar Pandey
Assistant Professor
Central Department of Computer Science
and Information Technology
Tribhuvan University
Kathmandu, Nepal

---

**Head of the Department**

Asst. Prof. Dr. Dhiraj Kedar Pandey
Central Department of Computer Science
and Information Technology
Tribhuvan University
Kathmandu, Nepal

**OCTOBER, 2026**

# Acknowledgements

First and foremost, I extend my deepest appreciation to my supervisor, **Asst. Prof. Dr. Dhiraj Kedar Pandey**, for their continuous guidance, motivation, and invaluable feedback throughout this research. Their insightful suggestions and constructive criticism provided during every stage of this work were instrumental in shaping the direction and quality of this thesis.

I am grateful to the Central Department of Computer Science and Information Technology, Institute of Science and Technology, Tribhuvan University, for providing the academic environment and resources necessary for this research.

Finally, I express my heartfelt gratitude to my family and friends for their unwavering support, patience, and encouragement throughout my graduate studies.

---

Erize Rai
OCTOBER, 2026

# Abstract

This thesis investigates the comparative effectiveness of self-supervised video encoder and vision-language model for video action recognition. Two state-of-the-art foundation models representing fundamentally different learning paradigms were evaluated: V-JEPA 2 (654M parameters), a self-supervised video encoder from Meta AI trained via masked spatio-temporal prediction in latent space, and Qwen2.5-VL (7B parameters), an autoregressive vision-language model from Alibaba Cloud. A frozen-feature evaluation protocol was employed on two complementary benchmarks: FineGym (4-class fine-grained gymnastics classification, 1,200 clips) and HMDB51 (51-class coarse-grained action recognition, 6,754 videos). The extracted representations were evaluated across nine classifiers, including three classical machine learning algorithms, five neural network classifiers, and a novel HAF-STD Net (Hierarchical Adaptive Fusion with Spectral-Temporal Decomposition) architecture designed to exploit multi-layer latent representations. On FineGym, V-JEPA 2 paired with HAF-STD Net achieved 98.1% accuracy and 97.8% macro F1, outperforming Qwen2.5-VL's best result (89.1% accuracy, 88.4% macro F1 with DeepMLP) by a 9.4 percentage point F1 margin. On HMDB51, V-JEPA 2 with HAF-STD Net achieved 84.7% accuracy and 85.3% macro F1, surpassing Qwen2.5-VL's best result (77.1% accuracy, 79.1% macro F1 with DeepMLP) by a 6.2 percentage point F1 margin. The results demonstrate that task-specialized self-supervised video encoders produce substantially more discriminative temporal representations than general-purpose VLMs, despite operating with an order of magnitude fewer parameters. Furthermore, the proposed HAF-STD Net shows that decomposing multi-layer latent representations into hierarchical appearance, motion, and depth-velocity streams through adaptive routing yields significant performance gains for frozen self-supervised video backbones.

*Keywords: Video Understanding, Self-Supervised Learning, Vision-Language Models, V-JEPA 2, Qwen2.5-VL, Action Recognition, HAF-STD Net*

# शोधसार

यस शोधप्रबन्धमा भिडियो कार्य पहिचानका लागि स्व-निरीक्षित भिडियो इन्कोडर र दृष्टि-भाषा मोडेलको तुलनात्मक प्रभावकारिता अध्ययन गरिएको छ। मौलिक रूपमा भिन्न शिक्षण प्रतिमान प्रतिनिधित्व गर्ने दुई अत्याधुनिक आधारभूत मोडेलहरू मूल्याङ्कन गरिएका छन्: V-JEPA 2 (६५.४ करोड प्यारामिटर) र Qwen2.5-VL (७ अर्ब प्यारामिटर) FineGym र HMDB51 बेन्चमार्कहरूमा फ्रोजन-फिचर मूल्याङ्कन प्रोटोकल प्रयोग गरिएको छ। निकालिएका प्रतिनिधित्वहरू नौ वर्गीकरणकर्तामा मूल्याङ्कन गरिएका छन्, जसमा बहु-तह अव्यक्त प्रतिनिधित्वहरू उपयोग गर्न डिजाइन गरिएको नवीन HAF-STD Net समावेश छ। FineGymमा, HAF-STD Netसँग V-JEPA 2ले ९८.१% Accuracy र ९७.८% Macro F1 हासिल गऱ्यो, जसले Qwen2.5-VLलाई ९.४ प्रतिशत बिन्दुले पछि पाऱ्यो। HMDB51मा, V-JEPA 2ले ८४.७% Accuracy र ८५.३% Macro F1 हासिल गऱ्यो, जसले Qwen2.5-VLलाई ६.२ प्रतिशत बिन्दुले पछि पाऱ्यो। नतिजाहरूले देखाउँछन् कि कार्य-विशिष्ट स्व-निरीक्षित भिडियो इन्कोडरहरूले सामान्य-उद्देश्यीय दृष्टि-भाषा मोडेलहरू भन्दा उल्लेखनीय रूपमा बढी विभेदकारी कालिक प्रतिनिधित्वहरू उत्पादन गर्छन्। यसबाहेक, प्रस्तावित HAF-STD Netले फ्रोजन स्व-निरीक्षित भिडियो ब्याकबोनहरूमा उल्लेखनीय कार्यसम्पादन सुधार गर्दछ।

*शब्द कुञ्जिका: Video Understanding, Self-Supervised Learning, Vision-Language Models, Action Recognition, HAF-STD Net*

# List of Abbreviations and Acronyms

| Abbreviation | Full Form |
|---|---|
| AdamW | Adam with Decoupled Weight Decay |
| AI | Artificial Intelligence |
| BB | Balance Beam |
| BERT | Bidirectional Encoder Representations from Transformers |
| BnB | BitsAndBytes (Quantization Library) |
| C3D | 3D Convolutional Network |
| CLIP | Contrastive Language-Image Pre-Training |
| CNN | Convolutional Neural Network |
| CUDA | Compute Unified Device Architecture |
| DWT | Discrete Wavelet Transform |
| EMA | Exponential Moving Average |
| FPS | Frames Per Second |
| FX | Floor Exercise |
| GELU | Gaussian Error Linear Unit |
| GFLOPs | Giga Floating Point Operations Per Second |
| GPU | Graphics Processing Unit |
| HAF-STD | Hierarchical Adaptive Fusion with Spectral-Temporal Decomposition |
| HMDB | Human Motion DataBase |
| HOF | Histogram of Optical Flow |
| HOG | Histogram of Oriented Gradients |
| I3D | Inflated 3D Convolutional Network |
| iDT | Improved Dense Trajectories |
| IoST | Institute of Science and Technology |
| JEPA | Joint Embedding Predictive Architecture |
| LLM | Large Language Model |
| LM | Language Model |
| LSTM | Long Short-Term Memory |
| MAE | Masked Autoencoder |
| MCC | Matthews Correlation Coefficient |
| MIT | Master of Information Technology |
| MLP | Multi-Layer Perceptron |
| MRoPE | Multimodal Rotary Position Embedding |
| MViT | Multiscale Vision Transformer |

| Abbreviation | Full Form |
|---|---|
| NF4 | Normal Float 4-bit (Quantization Format) |
| NLP | Natural Language Processing |
| OOM | Out of Memory |
| PCA | Principal Component Analysis |
| RBF | Radial Basis Function |
| ReLU | Rectified Linear Unit |
| RNN | Recurrent Neural Network |
| RoPE | Rotary Position Embedding |
| SDPA | Scaled Dot-Product Attention |
| SSL | Self-Supervised Learning |
| SVM | Support Vector Machine |
| TU | Tribhuvan University |
| UB | Uneven Bars |
| V-JEPA | Video Joint Embedding Predictive Architecture |
| VideoMAE | Video Masked Autoencoder |
| ViT | Vision Transformer |
| VLM | Vision-Language Model |
| VRAM | Video Random Access Memory |
| VT | Vault |

# List of Symbols

| Symbol | Description |
|---|---|
| $\mathbf{x}$ | Input Video Clip / Spatio-Temporal Frame Sequence |
| $\mathbf{z}$ | Latent Feature Representation Vector |
| $\mathbf{y}$ | Ground-Truth Action Class Label |
| $\hat{\mathbf{y}}$ | Predicted Action Class Label Probability Distribution |
| $\mathcal{L}_{\text{CE}}$ | Cross-Entropy Loss Function |
| $\mathcal{L}_{\text{Smooth}}$ | Label-Smoothing Cross-Entropy Loss Function |
| $\epsilon$ | Label-Smoothing Hyperparameter ($\epsilon = 0.1$) |
| $\alpha$ | Mixup Interpolation Beta Distribution Parameter ($\alpha = 0.2$) |
| $\lambda$ | Weight Decay / Regularization Parameter ($\lambda = 10^{-4}$) |
| $\eta$ | Learning Rate |
| $\kappa$ | Cohen's Kappa Coefficient |
| $\sigma(\cdot)$ | Softmax / Sigmoid Activation Function |
| $\mathbb{E}$ | Mathematical Expectation Operator |
| $d_{\text{embed}}$ | Embedding Dimension ($1280$ for V-JEPA 2, $3584$ for Qwen2.5-VL) |
| $N_{\text{frames}}$ | Number of Sampled Video Frames ($8$ for V-JEPA 2, $4$ for Qwen2.5-VL) |
| $C$ | Number of Target Action Classes ($4$ for FineGym, $51$ for HMDB51) |

# List of Tables

**Table 1:** Summary of evaluation datasets. . . . . 11
**Table 2:** Hardware configuration for experiments. . . . . 29
**Table 3:** Software libraries and their roles in the experimental pipeline. . . . . 30
**Table 4:** Architectural specifications of V-JEPA 2 and Qwen2.5-VL. . . . . 30
**Table 5:** Execution timeline for both dataset experiments. . . . . 35
**Table 6:** Classifier performance on FineGym — V-JEPA 2 features. . . . . 36
**Table 7:** Classifier performance on FineGym — Qwen2.5-VL features. . . . . 37
**Table 8:** FineGym: Comprehensive metric comparison (best classifiers). . . . . 38
**Table 9:** Classifier performance on HMDB51 — V-JEPA 2 features. . . . . 38
**Table 10:** Classifier performance on HMDB51 — Qwen2.5-VL features. . . . . 39
**Table 11:** HMDB51: Comprehensive metric comparison (best classifiers). . . . . 39

# List of Figures

**Figure 1:** Architecture of V-JEPA 2.1 self-supervised pretraining framework showing the dual x-Encoder and y-Encoder pipeline with multi-level fusion, mask-denoising prediction, and stop-gradient targets (Mur-Labadia et al., 2026). . . . . 14
**Figure 2:** Architecture of Qwen2.5-VL vision-language model illustrating the Vision Encoder with Conv3D patch embedding, window attention, and native resolution input processing, feeding into the Qwen2.5 LM Decoder with MRoPE temporal alignment (Qwen Team, 2025). . . . 17
**Figure 3:** Methodology flowchart showing the experimental pipeline. . . . . 18
**Figure 4:** Architecture of the proposed HAF-STD Net classifier. . . . . 23

# Contents

**Declaration** i

**Recommendation** ii

**Letter of Approval** iii

**Acknowledgements** iv

**Abstract** v

शोधसार vi

**List of Abbreviations and Acronyms** vii

**List of Symbols** ix

**List of Tables** x

**List of Figures** xi

**Table of Contents** xii

**1 Introduction** 1

1.1 General Introduction and Background . . . . . 1

1.2 Objectives . . . . . 2

1.2.1 General Objective . . . . . 2

1.2.2 Specific Objectives . . . . . 2

**2 Literature Review** 3

2.1 Theoretical Framework . . . . . 3

2.2 Literature Review . . . . . 4

2.3 Research Gap and Motivation . . . . . 8

**3 Materials and Methods** 10

3.1 Materials . . . . . 10

3.1.1 Benchmark Datasets . . . . . 10

3.1.2 Vision Foundation Models . . . 11
3.2 Methods . . . 17
3.2.1 Experimental Pipeline and Flowchart . . . 17
3.2.2 Frozen-Feature Protocol . . . 19
3.2.3 Feature Preprocessing and PCA Dimensionality Reduction . . . 19
3.2.4 Classifier Architectures . . . 20
3.2.5 Training Techniques and Optimization . . . 23
3.2.6 Evaluation Metrics . . . 24
3.3 Tools and Techniques . . . 29
3.3.1 Hardware Environment . . . 29
3.3.2 Software Stack . . . 30
3.3.3 Model Specifications . . . 30
3.4 Implementation Details . . . 31
3.4.1 FineGym Data Pipeline . . . 31
3.4.2 HMDB51 Data Pipeline . . . 31
3.4.3 Feature Extraction Pipeline . . . 32
3.4.4 Classifier Training Pipeline . . . 33
3.4.5 Memory Management and Checkpointing . . . 34
3.4.6 Data Pipeline Challenges . . . 34
3.4.7 Execution Timeline . . . 35

**4 Results and Discussion** **36**
4.1 Findings and Observations . . . 36
4.1.1 Experiment 1: FineGym (4-Class Fine-Grained Classification) . . . 36
4.1.2 Experiment 2: HMDB51 (51-Class Coarse-Grained Classification) 38
4.2 Result Analysis . . . 40
4.2.1 V-JEPA 2 Consistently Outperforms Qwen2.5-VL . . . 40
4.2.2 Performance Gap Depends on Task Granularity . . . 40
4.2.3 Parameter Efficiency . . . 40

**5 Conclusion and Recommendations** **41**
5.1 Conclusions . . . 41
5.2 Recommendations for Further Work . . . 42

**References** **45**

**A Source Code Listings** **46**
A.1 Data Acquisition . . . 46
A.2 Feature Extraction . . . 47
A.3 Classifier Architectures . . . 48

A.4 HAF-STD Net Implementation . . . . . 49

# Chapter 1

# Introduction

## 1.1 General Introduction and Background

Video understanding represents one of the most challenging frontiers in computer vision and artificial intelligence research. Unlike static image recognition, which has seen remarkable progress over the past decade, video comprehension demands that models capture not only spatial features within individual frames but also the complex temporal dynamics that unfold across sequences of frames (Bardes et al., 2024). This temporal dimension introduces an exponential increase in computational complexity and representational challenges, making video understanding a substantially more demanding task than its image-based counterpart.

Action recognition—the task of identifying human activities from video data—is a central problem in video understanding with broad applications spanning sports analytics, surveillance, human-computer interaction, healthcare monitoring, and content-based video retrieval (Bordes et al., 2024). The difficulty of action recognition varies dramatically depending on the granularity of the target classes: coarse-grained recognition of clearly distinct activities (e.g., "running" vs. "swimming") differs fundamentally from fine-grained recognition of visually similar activities (e.g., distinguishing between gymnastics events that occur in similar arenas).

Two fundamentally different paradigms have emerged for learning video representations in recent years. The first is self-supervised learning (SSL), where models learn representations from unlabeled video data by solving pretext tasks such as predicting masked regions in latent feature space (Assran et al., 2023, 2025). The second is vision-language modelling (VLM), where models are trained on large-scale paired image/video and text data using autoregressive language modelling objectives (Bai et al., 2025; Bordes et al., 2024). Both paradigms have produced models that achieve impressive results on a wide range of benchmarks, yet their relative merits across different recognition granularities remain insufficiently explored.

The rise of foundation models has intensified this question: self-supervised video encoders such as V-JEPA 2 (Assran et al., 2025) are trained exclusively on visual

data, learning representations through masked spatio-temporal prediction without any linguistic supervision. In contrast, vision-language models like Qwen2.5-VL (Bai et al., 2025) are trained on massive corpora of paired image-text and video-text data, acquiring both visual and semantic understanding through autoregressive generation. Whether the additional linguistic grounding in VLMs translates to better video representations for discriminative tasks—or whether it introduces noise that dilutes task-relevant temporal features—remains an open empirical question.

This thesis addresses this gap by presenting a systematic comparative analysis of a leading model from each paradigm: V-JEPA 2 (Assran et al., 2025), a self-supervised video encoder developed by Meta AI with 654 million parameters, and Qwen2.5-VL (Bai et al., 2025), a vision-language model from Alibaba Cloud with 7 billion parameters. Crucially, the comparison is conducted on two complementary benchmark datasets: (1) FineGym, a fine-grained gymnastics event classification benchmark with 4 classes, and (2) HMDB51, a large-scale general action recognition benchmark with 51 action classes. This dual-dataset evaluation enables assessment of how each paradigm performs across different recognition granularities.

## 1.2 Objectives

### 1.2.1 General Objective

- To conduct a rigorous comparative analysis of self-supervised and vision-language video representations for action recognition.

### 1.2.2 Specific Objectives

- To evaluate and compare the effectiveness of self-supervised video representations (V-JEPA 2) and vision-language representations (Qwen2.5-VL) for video action recognition across fine-grained (FineGym) and coarse-grained (HMDB51) task granularities.
- To design, implement, and evaluate a specialized classification architecture HAF-STD Net (Hierarchical Adaptive Fusion with Spectral-Temporal Decomposition) tailored for frozen self-supervised video representations.

# Chapter 2

# Literature Review

## 2.1 Theoretical Framework

Video action recognition has evolved from hand-crafted spatiotemporal feature extractors to deep convolutional neural networks and self-supervised Vision Transformers. Early approaches relied on dense visual trajectory features such as Histogram of Oriented Gradients (HOG), Histogram of Optical Flow (HOF), and Improved Dense Trajectories (iDT), which captured local motion dynamics but struggled with long-range temporal modeling. The application of deep learning to video introduced 3D CNN architectures, including C3D and Inflated 3D ConvNets (I3D), which extended 2D kernels into 3D space to learn joint spatiotemporal features. I3D demonstrated that inflating ImageNet-pretrained weights enabled strong action classification performance on Kinetics and HMDB51. Further architectural refinements, such as SlowFast Networks, introduced dual-rate sampling pathways to decouple spatial semantics from rapid temporal motion.

While supervised video models achieve high accuracy, they depend heavily on expensive manual annotations. Self-supervised learning (SSL) resolves this limitation by learning representations from unlabelled video sequences via predictive pretext tasks. Generative video models, such as VideoMAE, train Vision Transformers (ViTs) to reconstruct masked pixel regions at high masking ratios (up to 90%). Although VideoMAE achieves strong data efficiency, pixel-space reconstruction causes models to allocate representational capacity to high-frequency visual textures rather than high-level semantic action dynamics.

To overcome pixel-reconstruction overhead, Joint Embedding Predictive Architectures (JEPA) perform non-generative prediction directly in a learned latent feature space. Assran et al. (2023) introduced Image-JEPA (I-JEPA), demonstrating that predicting missing visual target embeddings from context patches produces highly semantic representations. Extending JEPA to temporal sequences, Bardes et al. (2024) proposed V-JEPA, which predicts spatiotemporal feature blocks in latent space. Assran et al. (2025) scaled video latent prediction with V-JEPA 2, a 654-million parameter

ViT-Huge encoder trained on extensive video datasets without pixel reconstruction or language supervision. The V-JEPA 2 training objective performs *mask-denoising prediction in representation space*: given two views $x$ and $y$ of a video, where patches have been randomly dropped from $y$, the encoder $E_\theta(\cdot)$ extracts representations from the unmasked view and a predictor $P_\phi(\cdot)$ predicts the masked representations by minimizing the $L_1$ distance to the stop-gradient target encoder output (Equation 3.4 in Chapter 3). The target encoder weights $\bar{\theta}$ are updated as an Exponential Moving Average of the context encoder (Equation 3.2), and a learnable mask token $\Delta_y$ indicates the positions of dropped patches. Both the encoder and predictor are parameterized as Vision Transformers with Rotary Position Embedding (RoPE).

Most recently, Mur-Labadia et al. (2026) introduced V-JEPA 2.1, which refines the training objective with a *dense per-token prediction loss* (Equation 3.5) and hierarchical intermediate supervision across encoder layers. Rather than computing a single global loss, V-JEPA 2.1 applies supervision individually to each masked spatiotemporal token, combined with a distance-weighted context loss on visible tokens (Equation 3.6). This dense formulation, combined with multi-layer intermediate supervision across encoder depth (Equation 3.8), significantly improves spatial and temporal representation grounding. The complete mathematical formulation of both V-JEPA 2 and V-JEPA 2.1 objectives is presented in Section 3.1.

Parallel to self-supervised feature learning, Vision-Language Models (VLMs) have emerged as powerful multimodal foundation models. As surveyed by Bordes et al. (2024) and Y. Zhang et al. (2023), VLMs connect visual encoders with autoregressive language decoders via visual instruction tuning. State-of-the-art vision-language models, such as Qwen2.5-VL (Bai et al., 2025) and Qwen3-VL (Qwen Team, 2025), feature dynamic spatiotemporal tokenization, native multi-frame processing, and long-context multimodal understanding. To enable practical deployment under strict hardware resource boundaries, Pathak and Han (2026) reviewed quantization, pruning, and low-bit inference techniques for large vision-language models.

## 2.2 Literature Review

The progression from Image-JEPA to V-JEPA 2.1 reveals a systematic trajectory of architectural refinement aimed at producing increasingly expressive latent representations. Assran et al. (2023) first established the foundational principle that predicting target patch representations from context patches—without pixel-level reconstruction—produces semantically richer embeddings than generative or contrastive alternatives. Their Image-JEPA demonstrated that a non-generative latent predictive objective avoids wasting model capacity on perceptually irrelevant high-frequency detail, while simultaneously eliminating the dependency on hand-crafted data augmentations

inherent to contrastive learning pipelines.

Building on this image-level foundation, Bardes et al. (2024) extended the joint-embedding predictive paradigm to the spatiotemporal domain with V-JEPA, introducing masked spatiotemporal block prediction in video feature space. The critical insight was that temporal coherence across video frames provides an especially rich supervisory signal for latent prediction, as adjacent frames share semantic content while exhibiting continuous motion dynamics. Assran et al. (2025) then scaled the V-JEPA framework significantly with V-JEPA 2, employing a ViT-Huge backbone with 654 million parameters trained on large-scale unlabelled video data. This scaling demonstrated that increasing both model capacity and training data volume within the JEPA paradigm yields representations that transfer competitively across diverse downstream tasks.

The most recent architectural advancement, V-JEPA 2.1 (Mur-Labadia et al., 2026), introduced two critical innovations: a dense per-token prediction loss that supervises each masked spatiotemporal token individually, and multi-layer intermediate supervision applied across encoder depth. These modifications fundamentally change the nature of learned representations by ensuring that intermediate encoder layers also acquire strong semantic features, rather than concentrating representational quality only in the final layer. This property is especially relevant to the present study, as it motivates the extraction of multi-layer features for the proposed HAF-STD Net architecture. In parallel, Y. Huang (2026) proposed Variational JEPA, which reformulates latent prediction as a probabilistic inference problem by introducing distribution uncertainty modelling, demonstrating that JEPA-style architectures can be extended beyond deterministic prediction to support stochastic control and predictive world modeling.

Integrating JEPA principles into multimodal and language domains represents an active frontier of research. Chen et al. (2026) proposed VL-JEPA, extending joint-embedding predictive learning to multimodal vision-language alignment without relying on autoregressive token generation. By performing cross-modal latent prediction between visual and textual representations, VL-JEPA demonstrated that the non-generative JEPA objective can bridge modality gaps while preserving the computational advantages of avoiding pixel or token reconstruction. H. Huang et al. (2025) combined Large Language Models with JEPA, demonstrating latent-space predictive loss objectives for language understanding. Their LLM-JEPA framework showed that large language models benefit from predictive representation learning in ways that complement standard autoregressive training. Liu et al. (2026) presented JEPA-Reasoner, decoupling latent-space reasoning from discrete token generation to enable unconstrained reasoning paths, while H. Zhang et al. (2026) introduced ThinkJEPA, empowering latent world models with large vision-language reasoning capabilities. Collectively, these works indicate a growing trend toward unifying

self-supervised prediction with language grounding, though none directly evaluate how these latent representations compare against instruction-tuned VLMs for discriminative video understanding tasks.

Beyond visual recognition, JEPA architectures serve as foundational world models for physical reasoning and decision-making. Bagatella et al. (2025) developed Temporal Difference JEPA, proving that latent-predictive representations enable zero-shot reinforcement learning across complex control environments. By combining temporal difference learning with JEPA's latent prediction, TD-JEPA demonstrated that forward predictive representations in a learned feature space encode sufficient dynamical information for effective policy transfer without environment-specific fine-tuning. Terver and Yang (2026) investigated the core drivers of success in physical planning using joint-embedding predictive world models, demonstrating strong generalization in physical simulation tasks. Their analysis identified that the latent prediction objective, rather than model scale alone, is the primary contributor to planning performance, providing evidence that JEPA representations capture causal-temporal structure that is qualitatively different from the associative representations learned through vision-language pretraining.

A fundamental distinction exists between the representation learning strategies of JEPA-based self-supervised models and instruction-tuned vision-language models. Self-supervised JEPA encoders such as V-JEPA 2.1 (Mur-Labadia et al., 2026) learn visual representations through prediction in latent space, entirely without textual supervision. Their learned features are inherently visual-geometric, capturing motion trajectories, spatial configurations, and temporal dynamics as distributed patterns within a continuous embedding space. In contrast, vision-language models such as Qwen2.5-VL (Bai et al., 2025; Qwen Team, 2025) acquire visual representations through cross-modal alignment with natural language, producing features that are linguistically grounded and semantically structured according to textual category boundaries. As discussed in the comprehensive survey by Bordes et al. (2024), this linguistic grounding provides VLMs with strong zero-shot generalization capabilities, since visual concepts are inherently linked to compositional language descriptions. Recent efforts have sought to bridge the gap between linguistic and geometric representations; notably, Lin et al. (2026) introduced Qwen-3D, a generalist 3D vision-language model built upon the Qwen backbone that augments visual tokens with 3D Rotary Positional Embeddings and operates attention directly in 3D scene space, enabling scalable cross-view and temporal reasoning through explicit geometric awareness rather than purely linguistic alignment. However, Y. Zhang et al. (2023) noted that vision-language alignment may bias representations toward semantically salient concepts that are easily described in natural language, potentially sacrificing fine-grained visual distinctions that lack clear linguistic labels.

The efficiency dimension further differentiates these paradigms. V-JEPA 2.1, with 654M parameters, achieves its representational power through architectural specialization and self-supervised pretraining on video data alone, whereas Qwen2.5-VL requires 7B parameters and massive multimodal corpora to achieve general-purpose visual understanding. Pathak and Han (2026) surveyed quantization and pruning strategies for deploying large VLMs under hardware constraints, highlighting that the parameter efficiency of specialized self-supervised encoders represents a significant practical advantage for resource-constrained deployment scenarios.

Downstream representation evaluation depends heavily on task granularity. The FineGym benchmark establishes that gymnastics event and sub-action classification requires capturing subtle kinematic body movements across visually similar apparatus environments. HMDB51 contains 6,754 clips across 51 diverse action categories, with supervised baselines ranging from 57.2% (iDT) to 80.7% (I3D) and 73.3% (VideoMAE).

The choice of downstream classifier significantly influences the evaluation of frozen representations. This study employs a diverse suite of classifiers spanning classical machine learning and modern neural network architectures. Support Vector Machines with Radial Basis Function kernels (Cortes & Vapnik, 1995) project features into high-dimensional spaces via the kernel trick, enabling non-linear decision boundaries and serving as strong baselines for feature evaluation. Random Forests (Breiman, 2001) construct ensembles of decision trees through bagging and random feature subsampling, providing robustness against overfitting and insensitivity to feature scaling. Logistic Regression (Cox, 1958) applies a linear classifier with L2 regularization, serving as a linear separability probe that reveals whether learned representations are directly discriminative without non-linear transformation.

Among neural network classifiers, Multi-Layer Perceptrons (MLPs) (Rumelhart et al., 1986) provide the foundational feedforward architecture with non-linear activation functions. Deeper variants with Batch Normalization (Ioffe & Szegedy, 2015) and Dropout (Srivastava et al., 2014) regularization enable training of deeper feedforward networks by stabilizing internal covariate shift and preventing co-adaptation of hidden units. Residual connections (He et al., 2016), originally introduced for deep convolutional networks, enable gradient flow through skip connections of the form $\mathbf{h}_{l+1} = \mathbf{h}_l + \mathcal{F}(\mathbf{h}_l, \mathbf{W}_l)$, allowing substantially deeper architectures to be trained without degradation. One-dimensional convolutional networks (LeCun et al., 1998) treat the feature dimension as a spatial axis, applying learned convolutional filters to capture local feature patterns and correlations. Finally, Transformer encoder blocks (Vaswani et al., 2017) employ multi-head self-attention to model global dependencies across feature dimensions, enabling each feature token to attend to all others through scaled dot-product attention.

## 2.3 Research Gap and Motivation

Despite significant advances in self-supervised video representation learning and multimodal foundation models, a comprehensive analysis of the existing literature reveals several critical research gaps that motivate the investigation undertaken in this thesis:

**Lack of Controlled Cross-Paradigm Comparison:** While self-supervised Joint Embedding Predictive Architectures (V-JEPA 2, Assran et al., 2025; V-JEPA 2.1, Mur-Labadia et al., 2026) and large vision-language models (Qwen2.5-VL, Bai et al., 2025; Qwen Team, 2025) have demonstrated remarkable individual capabilities, they have been evaluated exclusively within their own respective paradigms and benchmark suites. Self-supervised video encoders are typically assessed on masked-prediction benchmarks such as Kinetics-400 and Something-Something v2, whereas instruction-tuned vision-language models are primarily evaluated on visual question answering and text-generation tasks. To date, the literature lacks a direct, controlled empirical comparison of their frozen-feature representations under identical downstream classification conditions. Consequently, it remains an open question whether the visual-geometric features learned through latent-space predictive self-supervision or the linguistically grounded features acquired through multimodal instruction tuning provide superior transferability for video action understanding.

**Disparity Across Task Granularities:** Existing literature on video representation learning predominantly evaluates models on coarse-grained action recognition benchmarks such as HMDB51, where actions are characterized by distinct scenes, objects, and broad human movements. In contrast, fine-grained action recognition, exemplified by gymnastic routines in FineGym, requires distinguishing subtle kinematic differences, posture transitions, and limb trajectories in visually homogeneous environments. Whether the semantic abstractions of vision-language models or the temporal-predictive representations of JEPA encoders generalize more effectively across disparate levels of task granularity has not been systematically investigated.

**Under-Utilization of Multi-Layer Depth and Frequency Information:** Standard probing methodologies across both representation paradigms rely almost universally on simple global average pooling applied to the terminal encoder layer, followed by a linear classifier or shallow multi-layer perceptron. This conventional approach discards two vital sources of discriminative information. First, as established by Mur-Labadia et al. (2026), intermediate layers in deep video encoders encode rich hierarchical features that capture complementary spatiotemporal abstractions across network depth. Second, global spatial and temporal pooling collapses temporal dynamics into static summary statistics, failing to capture high-frequency motion transients and multi-scale temporal variations. Existing downstream probing architectures provide no specialized

mechanism to harness multi-layer depth representations alongside temporal frequency decomposition.

**Motivation and Contributions of this Study:** Motivated by these identified gaps, this thesis conducts the first rigorous, controlled empirical comparison between frozen representations extracted from V-JEPA 2 (654M parameters) and Qwen2.5-VL (7B parameters) across both fine-grained (FineGym) and coarse-grained (HMDB51) action recognition benchmarks. Both foundation models are systematically evaluated across a comprehensive suite of 11 diverse baseline classifiers, establishing fair and reproducible benchmark performance under identical protocol conditions. Furthermore, to address the architectural limitations of conventional probing, this study designs, implements, and evaluates the Hierarchical Adaptive Frequency Spatiotemporal Depth Network (HAF-STD Net), a novel classification architecture that leverages discrete wavelet multi-frequency decomposition, multi-depth feature extraction, and adaptive routing to maximize downstream spatiotemporal discrimination.

# Chapter 3

# Materials and Methods

## 3.1 Materials

### 3.1.1 Benchmark Datasets

Two complementary action recognition datasets were selected to evaluate representation quality at different levels of semantic granularity: FineGym for fine-grained, visually similar action categories, and HMDB51 for coarse-grained, diverse human actions.

#### FineGym

FineGym (Shao et al., 2020) is a large-scale, hierarchically annotated video dataset introduced at CVPR 2020 specifically designed to advance fine-grained action understanding in the domain of artistic gymnastics. FineGym organizes gymnastic routines into a three-level semantic hierarchy:

1. Event Level (Coarsest): Identifies the apparatus category—Balance Beam (BB), Floor Exercise (FX), Uneven Bars (UB), and Vault (VT).

2. Set Level (Mid-level): Groups sub-actions into functionally related categories within each event (e.g., "leap-jump-hop," "beam-turns," "dismount").

3. Element Level (Finest): Assigns specific, fine-grained labels to individual sub-actions (e.g., a particular type of double salto backward tucked). Standard benchmarking subsets at this level include Gym99 (99 element classes) and Gym288 (288 element classes).

In this study, only the **event-level** annotations were used, treating the task as a 4-class classification problem. A balanced subset of 1,200 event clips (300 clips per apparatus category) was acquired from YouTube using `yt-dlp` with temporal trimming based on the official annotation timestamps. The event-level task was selected because it tests whether vision foundation models can distinguish between four apparatus routines that share substantial visual overlap—all featuring a single gymnast performing acrobatic

movements in a competition arena—requiring the model to capture subtle differences in apparatus context, body orientation, and movement dynamics.

#### HMDB51

HMDB51 (Kuehne et al., 2011) is a widely adopted benchmark for general human action recognition, introduced at ICCV 2011 by the Serre Lab at Brown University. The dataset contains 6,766 video clips distributed across 51 action categories, with a minimum of 101 clips per class. Videos were primarily collected from digitized movies and public video repositories (YouTube, the Prelinger archive, and Google Videos), providing substantial diversity in viewpoint, background clutter, camera motion, lighting conditions, and video quality.

The 51 action classes span five broad categories of human motion:

1. General Facial Actions: e.g., smiling, laughing, chewing.
2. Facial Actions with Object Manipulation: e.g., eating, drinking, smoking.
3. General Body Movements: e.g., clapping, climbing, jumping, running.
4. Body Movements with Object Interaction: e.g., brushing hair, catching, drawing sword, shooting a gun.
5. Body Movements with Human Interaction: e.g., fencing, hugging, kicking, punching.

HMDB51 serves as a complementary evaluation to FineGym: whereas FineGym tests fine-grained discrimination among visually similar classes, HMDB51 assesses the model's ability to handle a large number of semantically diverse action categories under challenging real-world conditions. The full dataset of 6,754 successfully loaded video clips (51 classes) was obtained as a 2.12 GB zip archive via the Hugging Face Hub (`jili5044/hmdb51`), with an 80/20 train/evaluation split applied during feature preprocessing.

**Table 1:** Summary of evaluation datasets.

| Dataset | Clips | Classes | Task Granularity | Feature Dimension |
|---|---|---|---|---|
| FineGym | 1,200 | 4 | Fine-Grained | 1280 (V-JEPA 2) / 3584 (Qwen2.5-VL) |
| HMDB51 | 6,754 | 51 | Coarse-Grained | 1280 (V-JEPA 2) / 3584 (Qwen2.5-VL) |

### 3.1.2 Vision Foundation Models

Two vision foundation models representing fundamentally different representation learning paradigms were selected for comparative evaluation: V-JEPA 2.1

(self-supervised latent prediction) and Qwen2.5-VL (instruction-tuned vision-language model).

### V-JEPA 2.1 (Meta AI)

V-JEPA 2.1 (Assran et al., 2025; Mur-Labadia et al., 2026) is a 654-million parameter self-supervised video encoder based on the Vision Transformer Huge (ViT-H) architecture. Unlike generative models that reconstruct raw pixels, V-JEPA 2.1 learns representations by predicting masked spatiotemporal features in a learned latent space, entirely without language supervision or pixel reconstruction.

**Spatiotemporal Tokenization.** Input video clips are uniformly sampled to $T$ frames (8 frames in this study), each resized to $224 \times 224$ pixels. Frames are partitioned into non-overlapping spatiotemporal *tubelets* of size $2 \times 16 \times 16$ (temporal $\times$ height $\times$ width) using a 3D convolutional patch embedding layer. Each tubelet is linearly projected to produce a $D$-dimensional token ($D = 1280$ for ViT-H), yielding a sequence of $N = \frac{T}{2} \times \frac{H}{16} \times \frac{W}{16}$ tokens per clip:

$$\mathbf{z}_0 = \text{Conv3D}(\mathbf{V}) + \mathbf{E}_{\text{pos}}, \quad \mathbf{z}_0 \in \mathbb{R}^{N \times D} \tag{3.1}$$

where $\mathbf{V} \in \mathbb{R}^{T \times H \times W \times 3}$ is the input video tensor and $\mathbf{E}_{\text{pos}}$ encodes positional information via 3D Rotary Position Embedding (RoPE), which injects explicit spatiotemporal coordinates into attention computations without fixed sinusoidal encodings.

**Dual-Encoder Architecture.** V-JEPA 2.1 employs a dual-encoder framework. Given two augmented views $x$ and $y$ of the same video clip, random spatiotemporal patches are masked from view $y$. The *context encoder* $E_\theta$ (also called the x-Encoder) processes the full unmasked view $x$, while the *target encoder* $E_{\bar{\theta}}$ (the y-Encoder) processes the complete view $y$ to produce target representations. The target encoder is not trained by gradient descent; instead, its weights $\bar{\theta}$ are updated as an Exponential Moving Average (EMA) of the context encoder weights:

$$\bar{\theta} \leftarrow \tau\bar{\theta} + (1 - \tau)\theta, \quad \tau \in [0.996, 1.0) \tag{3.2}$$

Both encoders are standard Vision Transformers consisting of $L$ layers of Multi-Head Self-Attention (MHSA) and Feed-Forward Network (FFN) blocks:

$$\mathbf{z}'_l = \text{MHSA}(\text{LN}(\mathbf{z}_{l-1})) + \mathbf{z}_{l-1}, \quad \mathbf{z}_l = \text{FFN}(\text{LN}(\mathbf{z}'_l)) + \mathbf{z}'_l \tag{3.3}$$

where $\text{LN}(\cdot)$ denotes Layer Normalization and the attention mechanism uses Scaled Dot-Product Attention (SDPA) computed in float16 precision for memory efficiency.

**Mask-Denoising Prediction Objective.** A predictor network $P_\phi$, also parameterized as a Vision Transformer, takes the context encoder output $E_\theta(x)$ along with learnable mask tokens $\Delta_y$ (indicating the positions of dropped patches) and predicts the target representations of the masked regions. The V-JEPA 2 training objective minimizes the $L_1$ distance between predicted and target representations:

$$\underset{\theta,\,\phi,\,\Delta_y}{\text{minimize}} \quad \|P_\phi(\Delta_y, E_\theta(x)) - \text{sg}(E_{\bar{\theta}}(y))\|_1 \tag{3.4}$$

where $\text{sg}(\cdot)$ is the stop-gradient operator that prevents representational collapse by blocking gradients through the target encoder. The loss is computed only over predictions corresponding to the masked patch positions.

**Dense Per-Token Prediction Loss (V-JEPA 2.1).** V-JEPA 2.1 (Mur-Labadia et al., 2026) refines the training objective with a *dense per-token prediction loss* that supervises each masked spatiotemporal token individually, rather than computing a single global loss:

$$\mathcal{L}_{\text{predict}} = \frac{1}{|M|} \sum_{i \in M} \|P_\phi(E_\theta(x), \Delta_y)_i - \text{sg}(E_{\bar{\theta}}(y)_i)\|_1 \tag{3.5}$$

where $M$ is the set of masked patch indices and the subscript $i$ denotes the $i$-th token position. Additionally, a distance-weighted *context loss* is applied to nearby visible tokens to enforce dense spatial grounding of context representations:

$$\mathcal{L}_{\text{context}} = \frac{1}{|C|} \sum_{j \in C} \lambda_d(j) \cdot \|E_\theta(x)_j - \text{sg}(E_{\bar{\theta}}(y)_j)\|_1 \tag{3.6}$$

where $C$ is the set of visible context token indices and $\lambda_d(j)$ is a distance-dependent weighting function that emphasizes tokens spatiotemporally proximate to masked regions. The total V-JEPA 2.1 objective combines both losses:

$$\mathcal{L}_{\text{V-JEPA 2.1}} = \mathcal{L}_{\text{predict}} + \lambda_d \cdot \mathcal{L}_{\text{context}} \tag{3.7}$$

**Multi-Layer Intermediate Supervision.** A critical innovation in V-JEPA 2.1 is the application of the prediction objective not only at the final encoder layer but at multiple intermediate depths. The predictor $P_\phi$ fuses features from selected encoder layers at approximately 25%, 50%, 75%, and 100% depth, ensuring that intermediate representations also acquire strong semantic features. This multi-layer supervision is directly exploited in the present study: features are extracted from these 4 intermediate layers with per-frame spatial mean pooling, producing a 4D tensor per clip:

$$\mathbf{F}_{\text{multi}} = [\bar{\mathbf{z}}_{L/4};\ \bar{\mathbf{z}}_{L/2};\ \bar{\mathbf{z}}_{3L/4};\ \bar{\mathbf{z}}_L] \in \mathbb{R}^{4 \times T' \times D} \tag{3.8}$$

where $\bar{\mathbf{z}}_l \in \mathbb{R}^{T' \times D}$ is the spatially mean-pooled representation at layer $l$ and $T'$ is the temporal token count after tubelet grouping. For baseline classifiers, the standard approach of global mean pooling over the final layer's output yields a single 1,280-dimensional embedding per clip: $\mathbf{f} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{z}_{L,i} \in \mathbb{R}^D$.

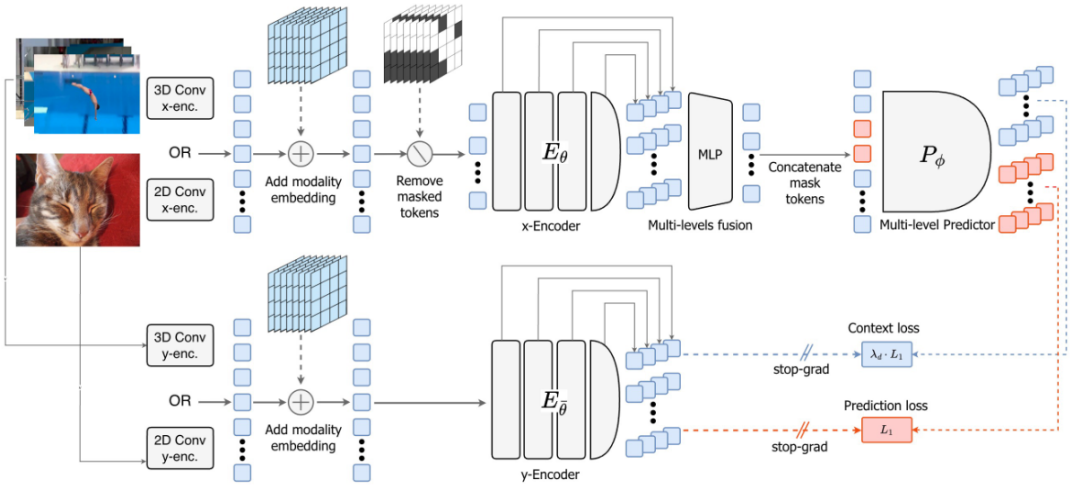



**Figure 1:** Architecture of V-JEPA 2.1 self-supervised pretraining framework showing the dual x-Encoder and y-Encoder pipeline with multi-level fusion, mask-denoising prediction, and stop-gradient targets (Mur-Labadia et al., 2026).

### Qwen2.5-VL (Alibaba Cloud)

Qwen2.5-VL (Bai et al., 2025; Qwen Team, 2025) is a 7-billion parameter autoregressive Vision-Language Model (VLM) that integrates visual understanding with language generation through cross-modal alignment. Unlike V-JEPA 2.1's purely visual self-supervised learning, Qwen2.5-VL acquires visual representations through joint training on massive image-text pairs and visual instruction tuning, producing features that are inherently grounded in linguistic semantics.

**Vision Encoder and Native Dynamic Resolution.** The Qwen2.5-VL Vision Encoder is a redesigned ViT trained from scratch to support *native dynamic resolution* processing. Rather than resizing all inputs to a fixed resolution (e.g., $224 \times 224$), the encoder preserves the original aspect ratio and resolution of each input frame, partitioning it into non-overlapping patches of size $14 \times 14$ pixels. For video inputs, a 3D convolutional (Conv3D) patch embedding layer groups adjacent frames temporally (temporal stride of 2) to form spatiotemporal patches, reducing the total token count while preserving temporal context:

$$\mathbf{v}_0 = \text{Conv3D}(\mathbf{V}_{\text{frames}}), \quad \mathbf{v}_0 \in \mathbb{R}^{N_v \times D_v} \tag{3.9}$$

where $N_v$ varies dynamically based on input resolution and $D_v$ is the vision encoder hidden dimension. This dynamic tokenization avoids information loss from aggressive

resizing and produces a variable number of visual tokens per input.

**Window Attention.** To manage computational cost for high-resolution inputs, the Vision Encoder employs *window attention* instead of global self-attention. Self-attention is computed locally within predefined spatial windows (e.g., $112 \times 112$ in patch space), reducing the computational complexity from $\mathcal{O}(N_v^2)$ to $\mathcal{O}(N_v \cdot w^2)$ where $w$ is the window size:

$$\text{WindowAttn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}_w \mathbf{K}_w^T}{\sqrt{d_k}}\right) \mathbf{V}_w \tag{3.10}$$

where subscript $w$ denotes restriction to tokens within the same spatial window. This enables efficient processing of long video sequences without quadratic memory growth.

**Multimodal Rotary Position Embedding (MRoPE).** A key architectural innovation in Qwen2.5-VL is the *Multimodal Rotary Position Embedding* (MRoPE), which extends the standard 1D RoPE used in language models to accommodate the multi-dimensional nature of visual inputs. The embedding dimension is decomposed into three segments, each encoding a separate positional axis—temporal ($t$), height ($h$), and width ($w$):

$$\text{MRoPE}(\mathbf{x}, t, h, w) = \mathbf{x} \odot [\cos(\theta_t), \cos(\theta_h), \cos(\theta_w)] + \hat{\mathbf{x}} \odot [\sin(\theta_t), \sin(\theta_h), \sin(\theta_w)] \tag{3.11}$$

where $\theta_t$, $\theta_h$, and $\theta_w$ are the rotation angles computed from the respective positional indices, and $\hat{\mathbf{x}}$ denotes the rotated complement of $\mathbf{x}$. For video inputs, the temporal position IDs are aligned with *absolute time* rather than frame indices, enabling the model to learn the pace of events and perform temporal grounding at second-level precision. Notably, when all three positional indices are set equal ($t = h = w$), MRoPE reduces to standard 1D RoPE, ensuring compatibility with pure text decoding.

**Visual-Language Fusion.** After the Vision Encoder processes visual inputs, the resulting visual tokens are projected into the language model's embedding space through a multi-layer perceptron (MLP) merger:

$$\mathbf{v}_{\text{lang}} = \text{MLP}_{\text{merge}}(\mathbf{v}_{\text{enc}}), \quad \mathbf{v}_{\text{lang}} \in \mathbb{R}^{N_v' \times D_{\text{LM}}} \tag{3.12}$$

where $D_{\text{LM}} = 3584$ is the hidden dimension of the Qwen2.5 language model decoder. The projected visual tokens are interleaved with text tokens to form a unified multimodal sequence, which is processed autoregressively by the Qwen2.5 LM Decoder. The model is trained to maximize the conditional log-likelihood of the next token:

$$\mathcal{L}_{\text{LM}} = -\sum_{t=1}^{T_{\text{seq}}} \log p_\theta(x_t \mid x_{<t}, \mathbf{v}_{\text{lang}}) \tag{3.13}$$

**4-bit NF4 Quantization.** Due to GPU memory constraints (15.6 GB VRAM on the NVIDIA Tesla T4), Qwen2.5-VL is loaded in 4-bit NormalFloat (NF4) quantization with double quantization via the BitsAndBytes library. NF4 quantization maps full-precision weights $\mathbf{W} \in \mathbb{R}$ to a 4-bit data type optimized for normally distributed neural network weights:

$$\mathbf{W}_{\text{NF4}} = \text{quantize}_{\text{NF4}}\left(\frac{\mathbf{W}}{s}\right) \cdot s, \quad s = \frac{\max(|\mathbf{W}_{\text{block}}|)}{q_{\max}} \tag{3.14}$$

where $s$ is a per-block scaling factor and $q_{\max}$ is the maximum quantile value in the NF4 data type. Double quantization further compresses the scaling factors themselves to 8-bit, reducing the model memory footprint from approximately 14 GB (float16) to approximately 4 GB while preserving inference quality.

**Feature Extraction Protocol.** In this study, Qwen2.5-VL is used as a frozen feature extractor rather than a generative model. Four uniformly sampled frames per clip are processed through the Vision Encoder and LM Decoder via a minimal chat template prompt. The hidden states from the final decoder layer are mean-pooled across the sequence dimension to produce a single 3,584-dimensional embedding per clip:

$$\mathbf{f}_{\text{Qwen}} = \frac{1}{T_{\text{seq}}} \sum_{t=1}^{T_{\text{seq}}} \mathbf{h}_t^{(L)}, \quad \mathbf{f}_{\text{Qwen}} \in \mathbb{R}^{3584} \tag{3.15}$$

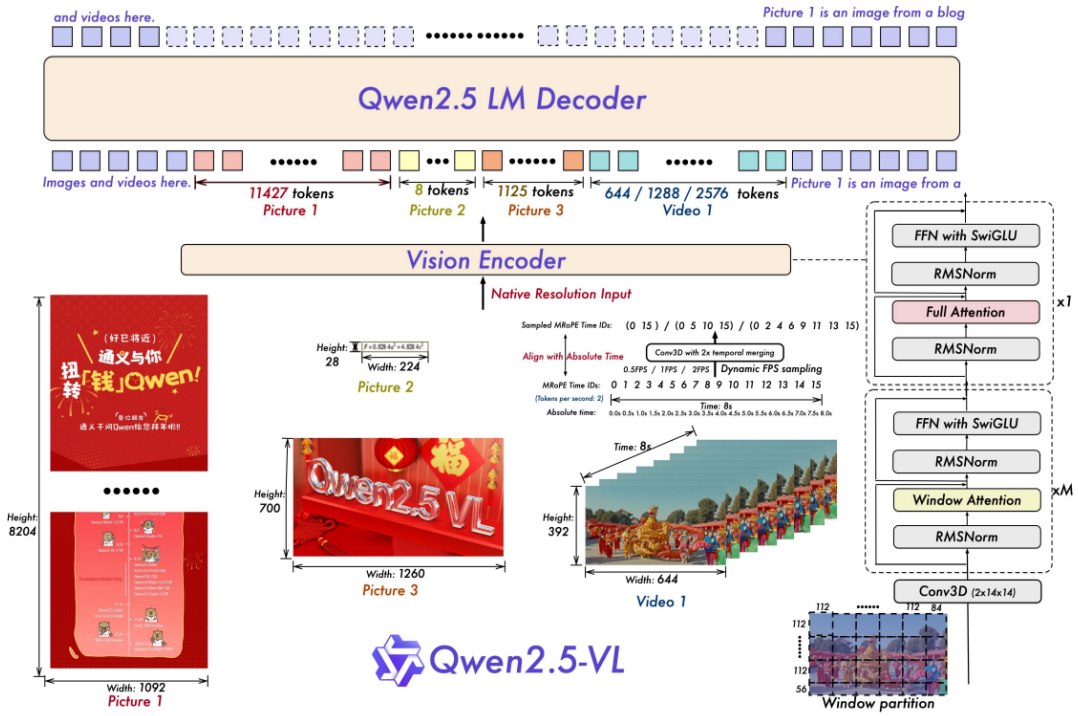



**Figure 2:** Architecture of Qwen2.5-VL vision-language model illustrating the Vision Encoder with Conv3D patch embedding, window attention, and native resolution input processing, feeding into the Qwen2.5 LM Decoder with MRoPE temporal alignment (Qwen Team, 2025).

## 3.2 Methods

### 3.2.1 Experimental Pipeline and Flowchart

The experimental pipeline follows a structured sequence: video frame extraction, multi-layer feature vector extraction, standardization, dimensionality reduction via Principal Component Analysis (PCA), and multi-classifier evaluation including the novel HAF-STD Net architecture. Figure 3 illustrates the pipeline.

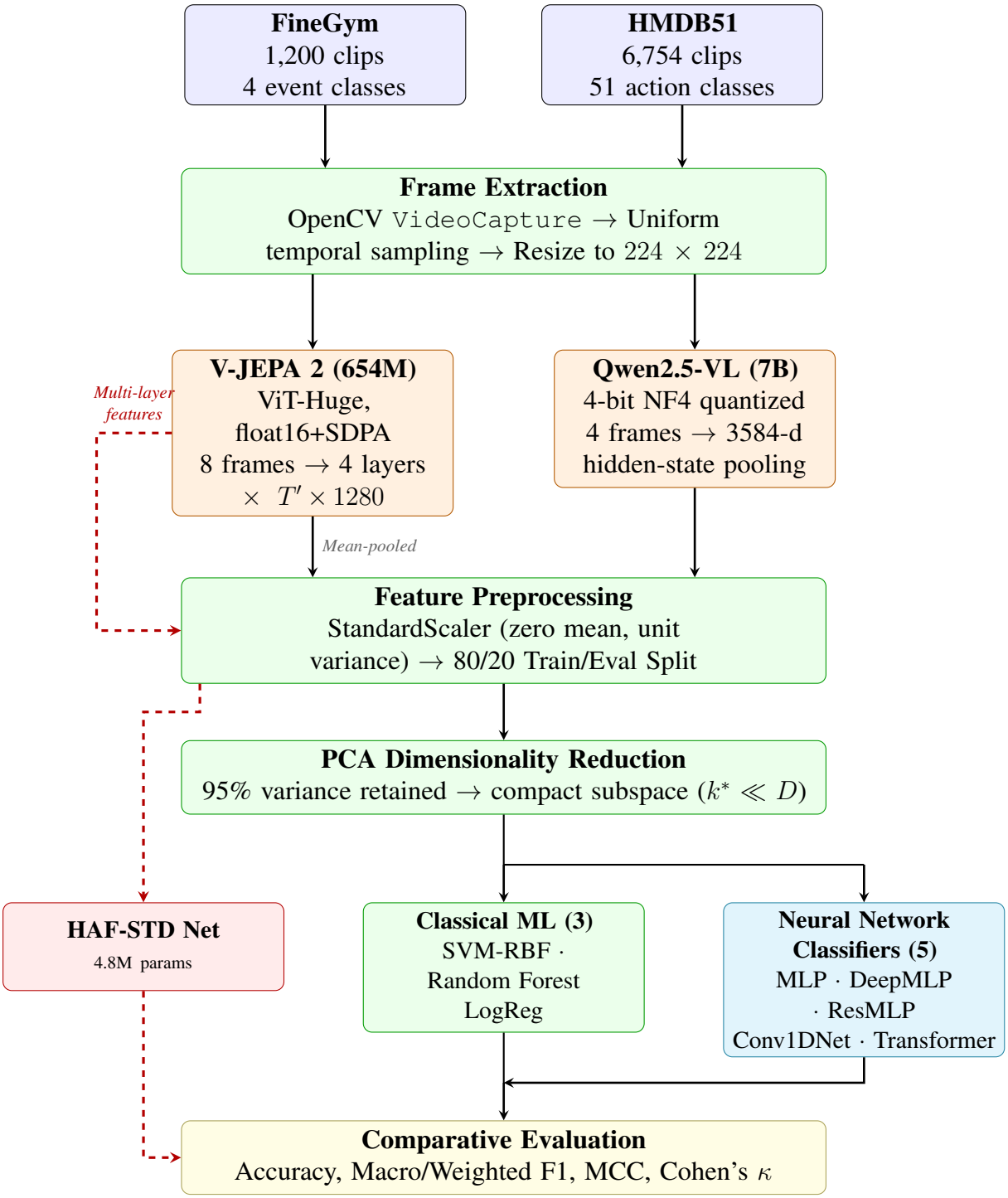



**Figure 3:** Methodology flowchart showing the experimental pipeline.

### 3.2.2 Frozen-Feature Protocol

Under the frozen-feature protocol (Assran et al., 2023, 2025), backbone model weights $\boldsymbol{\Theta}$ remain fixed throughout downstream training:

$$\mathbf{z}_i = f(\mathbf{x}_i; \boldsymbol{\Theta}_{\text{frozen}}), \quad \nabla_{\boldsymbol{\Theta}} \mathcal{L} = \mathbf{0} \tag{3.16}$$

Only downstream classifier weights $\mathbf{W}_{\text{head}}$ are optimized:

$$\mathbf{W}^*_{\text{head}} = \arg\min_{\mathbf{W}_{\text{head}}} \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}\left(g(\mathbf{z}_i; \mathbf{W}_{\text{head}}), y_i\right) \tag{3.17}$$

### 3.2.3 Feature Preprocessing and PCA Dimensionality Reduction

Extracted raw feature representations $\mathbf{Z} \in \mathbb{R}^{N \times D}$ are first standardized to zero mean and unit variance using the training-split statistics:

$$\tilde{\mathbf{z}}_{ij} = \frac{z_{ij} - \mu_j}{\sigma_j}, \quad \mu_j = \frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} z_{ij}, \quad \sigma_j = \sqrt{\frac{1}{N_{\text{train}} - 1} \sum_{i=1}^{N_{\text{train}}} (z_{ij} - \mu_j)^2} \tag{3.18}$$

where $j \in \{1, \ldots, D\}$ indexes feature dimensions. Standardization ensures that all features contribute equally to the subsequent dimensionality reduction step, preventing high-variance dimensions from dominating the principal component decomposition. The same $\mu_j$ and $\sigma_j$ statistics computed from the training split are applied to the test split to avoid data leakage.

**Principal Component Analysis (PCA)** is then applied to project the standardized features into a lower-dimensional, decorrelated subspace. Given the standardized feature matrix $\tilde{\mathbf{Z}} \in \mathbb{R}^{N \times D}$, PCA computes the covariance matrix:

$$\mathbf{C} = \frac{1}{N_{\text{train}} - 1} \tilde{\mathbf{Z}}^{\top}_{\text{train}} \tilde{\mathbf{Z}}_{\text{train}} \in \mathbb{R}^{D \times D} \tag{3.19}$$

and performs eigendecomposition $\mathbf{C} = \mathbf{V} \boldsymbol{\Lambda} \mathbf{V}^{\top}$, where $\boldsymbol{\Lambda} = \text{diag}(\lambda_1, \lambda_2, \ldots, \lambda_D)$ contains eigenvalues sorted in descending order ($\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_D \geq 0$) and $\mathbf{V} = [\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_D]$ contains the corresponding orthonormal eigenvectors (principal components). The number of retained components $k^*$ is selected as the minimum $k$ that preserves at least 95% of the total variance:

$$k^* = \arg\min_{k} \left( \frac{\sum_{j=1}^{k} \lambda_j}{\sum_{j=1}^{D} \lambda_j} \geq 0.95 \right) \tag{3.20}$$

The projection onto the retained subspace is then given by:

$$\mathbf{Z}_{\text{PCA}} = \tilde{\mathbf{Z}}\,\mathbf{V}_{:k^*} \in \mathbb{R}^{N \times k^*} \tag{3.21}$$

where $\mathbf{V}_{:k^*} \in \mathbb{R}^{D \times k^*}$ contains the first $k^*$ principal components.

The 95% variance retention threshold was chosen as a standard balance between information preservation and dimensionality reduction. On the FineGym dataset, PCA reduced V-JEPA 2 features from $D = 1{,}280$ to $k^* = 90$ dimensions and Qwen2.5-VL features from $D = 3{,}584$ to $k^* = 142$ dimensions. On HMDB51, V-JEPA 2 features were reduced to $k^* = 286$ and Qwen2.5-VL features to $k^* = 719$ dimensions. The substantially higher retained dimensionality for HMDB51 reflects the greater representational complexity required to distinguish 51 action classes compared to FineGym's 4 classes.

PCA serves three key purposes in this pipeline: (1) it decorrelates features, removing redundant collinear dimensions that arise from the over-parameterized encoder architectures; (2) it reduces computational cost for the classical ML classifiers (SVM-RBF, Random Forest), which scale poorly with high-dimensional inputs; and (3) it mitigates the curse of dimensionality, particularly for the smaller FineGym dataset where the sample-to-feature ratio would otherwise be unfavourable. Importantly, PCA is applied *only* to the baseline classifier pathway. The HAF-STD Net receives the full-dimensional, multi-layer temporal tensor without PCA reduction, as its spectral-temporal decomposition module (Section 3.2.4) operates directly on the spatiotemporal structure and benefits from preserving the complete representational capacity.

### 3.2.4 Classifier Architectures

Extracted features are evaluated across 8 baseline classifiers plus the HAF-STD Net architecture (9 classifiers total):

#### Classical Machine Learning Classifiers

1. **Support Vector Machine (SVM-RBF)** (Cortes & Vapnik, 1995)**:** Radial Basis Function kernel $K(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2\right)$ with $C = 1.0$.

2. **Random Forest** (Breiman, 2001)**:** Ensemble of 500 decision trees with balanced class weighting.

3. **Logistic Regression** (Cox, 1958)**:** Linear probe with L2 regularization ($C = 1.0$) and multinomial softmax.

**Neural Network Classifiers**

1. **Multi-Layer Perceptron (MLP)** (Rumelhart et al., 1986)**:** Scikit-learn MLP with 2 hidden layers (512, 256), ReLU, and Adam optimizer.

2. **DeepMLP:** 4-layer feedforward network (Input $\rightarrow$ 512 $\rightarrow$ 256 $\rightarrow$ 128 $\rightarrow$ Classes) with BatchNorm (Ioffe & Szegedy, 2015), ReLU, and Dropout (Srivastava et al., 2014) (0.3).

3. **ResMLP** (He et al., 2016)**:** Deep residual MLP with skip connections: $\mathbf{h}_{l+1} = \mathbf{h}_l + \mathcal{F}(\mathbf{h}_l, \mathbf{W}_l)$.

4. **Conv1DNet** (LeCun et al., 1998)**:** 1D Convolutional network projecting feature dimension as spatial channels followed by adaptive max pooling.

5. **MiniTransformer** (Vaswani et al., 2017)**:** Self-Attention transformer encoder block with multi-head attention ($h = 4$) and feedforward projection.

**HAF-STD Net Classifier**

We propose **HAF-STD Net** (Hierarchical Adaptive Fusion with Spectral-Temporal Decomposition), a specialized classification architecture that jointly exploits V-JEPA 2.1's unique properties of temporal coherence and deep self-supervision. The architecture contains 4.8M trainable parameters and operates on multi-layer per-frame features extracted from the frozen V-JEPA 2 encoder. HAF-STD Net introduces three core components:

**1. Spectral-Temporal Decomposition via Haar DWT.** A 1D Haar Discrete Wavelet Transform is applied along the temporal axis of V-JEPA 2's latent token sequences at each selected encoder layer. This decomposes the temporal signal into low-frequency (appearance/trend) and high-frequency (motion/change dynamics) components:

$$\mathbf{a}_k = \frac{1}{2}(\mathbf{h}_{2t} + \mathbf{h}_{2t+1}), \quad \mathbf{d}_k = \frac{1}{2}(\mathbf{h}_{2t} - \mathbf{h}_{2t+1}) \tag{3.22}$$

where $\mathbf{a}_k$ captures static appearance and $\mathbf{d}_k$ captures motion dynamics at layer $k$. The mean-pooled approximation and detail coefficients form the *appearance stream* and *motion stream*, respectively.

**2. Depth Representation Velocity.** Inter-layer deltas are computed between consecutive selected encoder layers:

$$\Delta\mathbf{H}_k = \bar{\mathbf{H}}_{k+1} - \bar{\mathbf{H}}_k \tag{3.23}$$

where $\bar{\mathbf{H}}_k$ denotes the temporally mean-pooled hidden state at layer $k$. These *velocity vectors* capture how the model's internal understanding progressively transforms across its depth—a signal that is action-class-dependent due to V-JEPA 2.1's deep self-supervision training.

**3. Adaptive Routing over Depth $\times$ Frequency Streams.** The three streams per layer (appearance, motion, velocity) are independently projected through dedicated linear heads, yielding $4 \times 3 = 12$ feature vectors. An adaptive routing network learns input-dependent fusion weights:

$$\mathbf{w} = \text{softmax}\left(\text{MLP}\left([\mathbf{s}_1; \mathbf{s}_2; \ldots; \mathbf{s}_{12}]\right)\right), \quad \mathbf{f} = \sum_{i=1}^{12} w_i \cdot \mathbf{s}_i \tag{3.24}$$

The fused representation $\mathbf{f}$ is classified via a cosine similarity head with learnable temperature scaling $\tau$:

$$p(y = c|\mathbf{f}) = \frac{\exp\left(\tau \cdot \hat{\mathbf{f}}^T \hat{\mathbf{w}}_c\right)}{\sum_{j=1}^{C} \exp\left(\tau \cdot \hat{\mathbf{f}}^T \hat{\mathbf{w}}_j\right)} \tag{3.25}$$

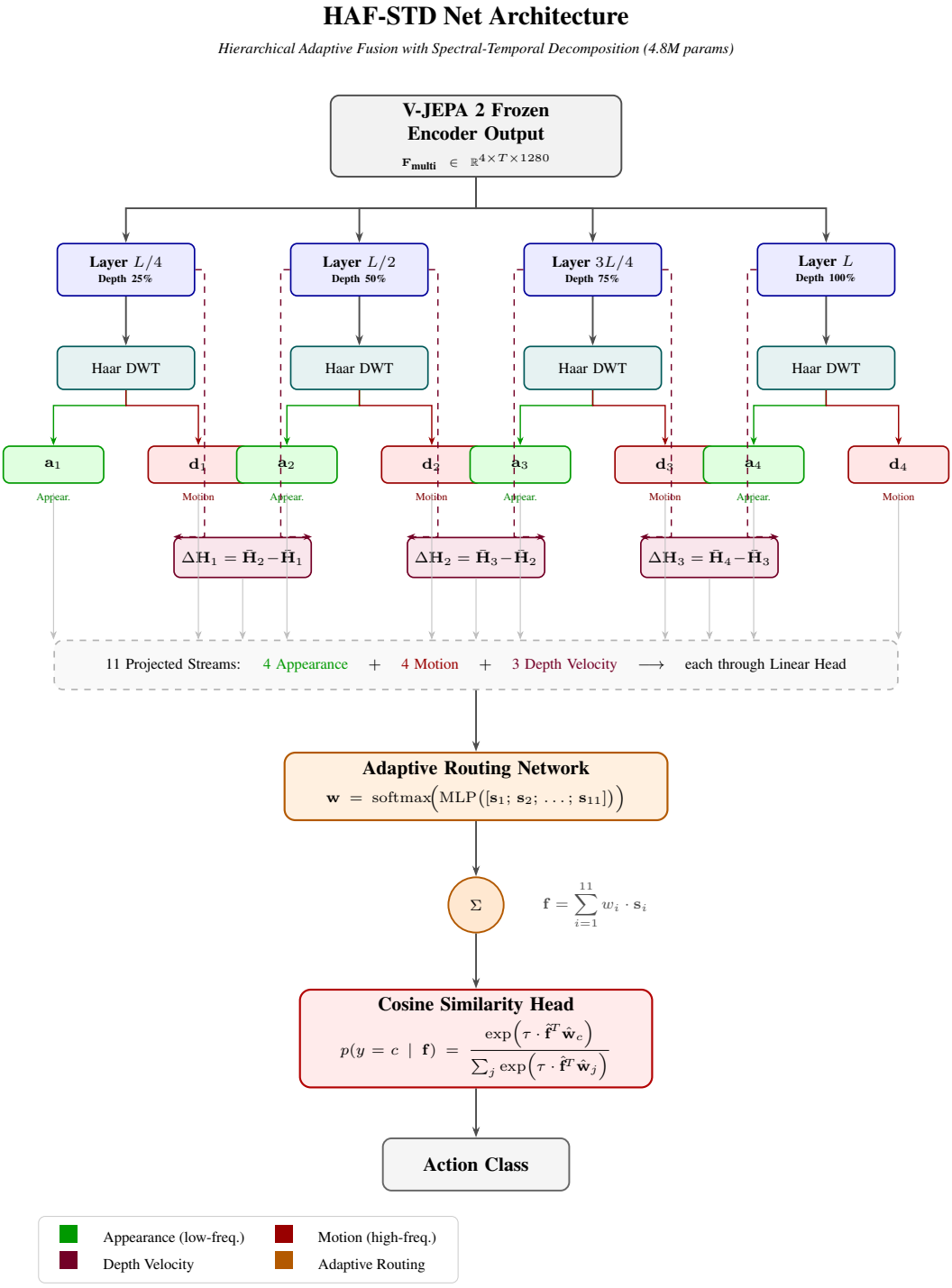



**Figure 4:** Architecture of the proposed HAF-STD Net classifier.

### 3.2.5 Training Techniques and Optimization

All neural network classifiers are trained using:

- **Label Smoothing** ($\epsilon = 0.1$)**:** Softens one-hot target vectors $y_c^{\text{smooth}} = (1 - \epsilon)y_c + \frac{\epsilon}{C}$.
- **Feature-Space Mixup** ($\alpha = 0.2$)**:** Interpolates feature pairs: $\tilde{\mathbf{z}} = \lambda \mathbf{z}_i + (1 - \lambda)\mathbf{z}_j$.

- **Cosine Annealing LR:** Learning rate schedule decaying from $10^{-3}$ to $10^{-6}$.
- **Optimizer:** AdamW with weight decay $\lambda = 10^{-4}$.
- **Early Stopping:** Triggers after 15 epochs without validation loss improvement.

### 3.2.6 Evaluation Metrics

To thoroughly assess classification performance and mitigate potential biases arising from class imbalance or superficial prediction heuristics, model evaluation is grounded on a comprehensive suite of quantitative metrics. In this study, experiments encompass both fine-grained action classification on FineGym ($C = 4$ sub-action categories) and coarse-grained classification across diverse motion categories on HMDB51 ($C = 51$ classes).

#### Confusion Matrix and Multi-Class Decomposition

All evaluated metrics are derived from the empirical confusion matrix $\mathbf{M} \in \mathbb{N}^{C \times C}$, where each element $M_{ij}$ denotes the count of test samples whose true ground-truth class is $i$ but were classified into class $j$ by the model. The total number of evaluation samples is given by $N = \sum_{i=1}^{C} \sum_{j=1}^{C} M_{ij}$.

For any individual target class $c \in \{1, 2, \ldots, C\}$, multi-class evaluation can be decomposed into a set of binary One-vs-Rest (OvR) classification problems using four fundamental quantities:

- **True Positives ($TP_c$):** The number of video instances correctly predicted as class $c$:

$$TP_c = M_{cc} \tag{3.26}$$

- **False Positives ($FP_c$):** The number of video instances belonging to classes other than $c$ that are incorrectly predicted as class $c$ (Type I error / false alarm):

$$FP_c = \sum_{i \neq c} M_{ic} \tag{3.27}$$

- **False Negatives ($FN_c$):** The number of video instances belonging to true class $c$ that are incorrectly assigned to a different class (Type II error / missed detection):

$$FN_c = \sum_{j \neq c} M_{cj} \tag{3.28}$$

- **True Negatives ($TN_c$):** The number of video instances not belonging to class $c$ that

are correctly predicted as non-$c$:

$$TN_c = \sum_{i \neq c} \sum_{j \neq c} M_{ij} = N - TP_c - FP_c - FN_c \tag{3.29}$$

The total ground-truth support (number of true instances) for class $c$ is defined as $N_c = TP_c + FN_c = \sum_{j=1}^{C} M_{cj}$, while the total number of predictions assigned to class $c$ is $P_c = TP_c + FP_c = \sum_{i=1}^{C} M_{ic}$.

### Classification Accuracy

Overall classification accuracy measures the fraction of all test video clips whose top predicted class label coincides with the ground-truth annotation:

$$\text{Accuracy} = \frac{\sum_{c=1}^{C} M_{cc}}{N} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}\left(\hat{y}_i = y_i\right) \tag{3.30}$$

where $\hat{y}_i = \arg\max_c \, p(y = c \mid \mathbf{x}_i)$ is the model's highest-probability predicted class for sample $\mathbf{x}_i$, $y_i$ is the corresponding ground-truth label, and $\mathbf{1}(\cdot)$ is the indicator function.

In single-label multi-class video classification, this is standard overall accuracy (top-1 decision rule). Under uniform random chance, the expected accuracy corresponds to $1/C$, yielding $25.0\%$ for FineGym ($C = 4$) and approximately $1.96\%$ for HMDB51 ($C = 51$). While overall accuracy provides an intuitive global benchmark of model correctness, it weights all instances equally and can thus be dominated by high-support majority classes. Consequently, additional class-balanced and chance-adjusted metrics are required.

### Precision (Macro and Weighted)

Precision assesses the reliability or purity of model predictions for a given category. For class $c$, it represents the probability that a sample predicted to be class $c$ truly belongs to class $c$:

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c} = \frac{M_{cc}}{\sum_{i=1}^{C} M_{ic}} \tag{3.31}$$

High precision indicates that when the classifier predicts an action, the prediction has a low rate of false alarms. To summarize precision across all $C$ classes, two aggregation schemes are reported:

- **Macro-Averaged Precision:** The unweighted arithmetic mean across all per-class precisions:

$$\text{Precision}_{\text{macro}} = \frac{1}{C} \sum_{c=1}^{C} \text{Precision}_c \tag{3.32}$$

Macro precision assigns equal importance to every class regardless of class support size, strictly highlighting models that misclassify smaller or more ambiguous classes.

- **Weighted Precision:** The support-weighted average of per-class precisions:

$$\text{Precision}_{\text{weighted}} = \sum_{c=1}^{C} \frac{N_c}{N} \text{Precision}_c \tag{3.33}$$

**Recall / Sensitivity (Macro and Weighted)**

Recall (also known as true positive rate or sensitivity) measures the completeness of detection for a given class. For class $c$, it indicates the proportion of true class $c$ video instances that the model successfully retrieves:

$$\text{Recall}_c = \frac{TP_c}{TP_c + FN_c} = \frac{M_{cc}}{\sum_{j=1}^{C} M_{cj}} = \frac{M_{cc}}{N_c} \tag{3.34}$$

High recall reflects that the model rarely misses actual instances of the specified action. The aggregated multi-class recall variants are computed as:

- **Macro-Averaged Recall:** The unweighted arithmetic average of per-class recalls:

$$\text{Recall}_{\text{macro}} = \frac{1}{C} \sum_{c=1}^{C} \text{Recall}_c \tag{3.35}$$

- **Weighted Recall:** The support-weighted average of class recalls:

$$\text{Recall}_{\text{weighted}} = \sum_{c=1}^{C} \frac{N_c}{N} \text{Recall}_c = \frac{\sum_{c=1}^{C} M_{cc}}{N} = \text{Accuracy} \tag{3.36}$$

In single-label multi-class evaluation, weighted recall is mathematically identical to overall classification accuracy.

**F1-Score (Macro and Weighted)**

Because precision and recall frequently exhibit an inverse trade-off, the F1-score provides a balanced harmonic mean between the two measures. For class $c$, it is defined as:

$$F1_c = 2 \cdot \frac{\text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c} = \frac{2TP_c}{2TP_c + FP_c + FN_c} \tag{3.37}$$

The harmonic mean ensures that if either precision or recall is low for a given category, the corresponding $F1_c$ is heavily penalized. Multi-class aggregation is performed as follows:

- **Macro F1-Score:** The unweighted arithmetic average of individual class F1-scores:

$$F1_{\text{macro}} = \frac{1}{C} \sum_{c=1}^{C} F1_c \tag{3.38}$$

  Throughout this thesis, Macro F1 is treated as the primary ranking metric because it treats every action category with equal priority. A model cannot achieve a high Macro F1 score by solely predicting dominant classes while neglecting fine-grained or difficult minority classes.

- **Weighted F1-Score:** The support-weighted mean of per-class F1-scores:

$$F1_{\text{weighted}} = \sum_{c=1}^{C} \frac{N_c}{N} F1_c \tag{3.39}$$

**Matthews Correlation Coefficient (MCC)**

The Matthews Correlation Coefficient (MCC) is widely considered one of the most reliable and statistically sound omnibus metrics for classification. Unlike accuracy or individual F1 formulations that only focus on subset cells of the confusion matrix, MCC utilizes all elements of the confusion matrix symmetrically. Originally formulated for binary classification, MCC was generalized to the multi-class case ($C > 2$) by Gorodkin through covariance analysis of categorical vectors:

$$\text{MCC} = \frac{c \cdot N - \sum_{k=1}^{C} P_k \, N_k}{\sqrt{\left(N^2 - \sum_{k=1}^{C} P_k^2\right) \left(N^2 - \sum_{k=1}^{C} N_k^2\right)}} \tag{3.40}$$

where:

- $c = \sum_{k=1}^{C} M_{kk}$ is the total number of correctly classified video clips,
- $N$ is the total number of test samples,
- $P_k = \sum_{i=1}^{C} M_{ik}$ is the total number of times class $k$ was predicted (column sum $k$),
- $N_k = \sum_{j=1}^{C} M_{kj}$ is the total number of ground-truth occurrences of class $k$ (row sum $k$).

MCC generates a correlation coefficient bounded strictly in the interval $[-1.0, +1.0]$:

- MCC $= +1.0$: Represents perfect prediction agreement where every test sample is correctly identified.
- MCC $= 0.0$: Represents prediction performance no better than random guessing based on marginal class distributions.

- MCC $= -1.0$: Represents complete inverse classification (total disagreement).

A fundamental advantage of MCC is that a trivial classifier (such as one that always predicts the majority class) yields an MCC of $0.0$, whereas overall accuracy would report misleadingly high scores.

### Cohen's Kappa ($\kappa$)

Cohen's Kappa ($\kappa$) is a chance-adjusted measure of inter-rater agreement adapted for machine learning classification, where the classifier's predictions and the true ground-truth annotations are treated as two independent raters:

$$\kappa = \frac{p_o - p_e}{1 - p_e} \tag{3.41}$$

where:

- $p_o$ is the observed empirical agreement, equivalent to overall classification accuracy:

$$p_o = \frac{1}{N} \sum_{c=1}^{C} M_{cc} \tag{3.42}$$

- $p_e$ is the hypothetical probability of agreement occurring purely by chance, computed from the product of marginal distribution proportions:

$$p_e = \frac{1}{N^2} \sum_{c=1}^{C} P_c \cdot N_c = \sum_{c=1}^{C} \left( \frac{\sum_{i=1}^{C} M_{ic}}{N} \right) \left( \frac{\sum_{j=1}^{C} M_{cj}}{N} \right) \tag{3.43}$$

The metric normalizes observed accuracy against the baseline chance level $p_e$. Values of $\kappa$ range between $-1.0$ and $+1.0$, typically interpreted according to the standard Landis and Koch agreement scale:

- $\kappa < 0.00$: Poor agreement (worse than random chance).
- $0.00 \leq \kappa \leq 0.20$: Slight agreement.
- $0.21 \leq \kappa \leq 0.40$: Fair agreement.
- $0.41 \leq \kappa \leq 0.60$: Moderate agreement.
- $0.61 \leq \kappa \leq 0.80$: Substantial agreement.
- $0.81 \leq \kappa \leq 1.00$: Almost perfect agreement.

While MCC originates from Pearson correlation and treats confusion errors symmetrically in continuous Euclidean vector space, Cohen's Kappa originates from categorical agreement statistics. Reporting both metrics alongside Macro F1 provides mutually reinforcing validation that observed performance gains reflect genuine representational superiority rather than statistical artifacts.

## 3.3 Tools and Techniques

### 3.3.1 Hardware Environment

**Table 2:** Hardware configuration for experiments.

| **Component** | **Specification** |
|---|---|
| GPU | NVIDIA Tesla T4 (15.6 GB VRAM) |
| CPU | Intel Xeon (2 cores) |
| RAM | 13 GB System Memory |
| Platform | Kaggle Notebooks |
| CUDA Version | 12.x |
| Persistent Storage | `/kaggle/working/` (saved as output) |

The NVIDIA Tesla T4 GPU, based on the Turing architecture, provides 15.6 GB of VRAM with support for mixed-precision (FP16) and INT8/INT4 operations. While not sufficient to load Qwen2.5-VL at full precision (which requires approximately 14 GB in float16), the T4 supports 4-bit quantization through the BitsAndBytes library, enabling deployment of the 7B-parameter model within the available memory budget.

### 3.3.2 Software Stack

**Table 3:** Software libraries and their roles in the experimental pipeline.

| Library | Version | Purpose |
|---|---|---|
| Python | 3.12.13 | Programming language |
| PyTorch | 2.x (Latest) | Deep learning framework, CUDA backend, custom NN classifiers |
| Transformers | Latest | Hugging Face model hub, V-JEPA 2 and Qwen2.5-VL loading |
| BitsAndBytes | Latest | 4-bit NF4 quantization for Qwen2.5-VL |
| scikit-learn | Latest | Classical ML classifiers, PCA, metrics |
| OpenCV | Latest | Video frame extraction (`cv2.VideoCapture`) |
| yt-dlp | Latest | YouTube video downloading with temporal trimming |
| huggingface_hub | Latest | HMDB51 dataset download (`hf_hub_download`) |
| NumPy | Latest | Numerical operations, frame indexing |
| Pillow (PIL) | Latest | Image format conversion |

### 3.3.3 Model Specifications

**Table 4:** Architectural specifications of V-JEPA 2 and Qwen2.5-VL.

| Specification | V-JEPA 2 | Qwen2.5-VL |
|---|---|---|
| Parameters | 654M | 7B |
| Architecture | ViT-Huge | Qwen2.5 + ViT |
| Paradigm | Self-supervised (JEPA) | Autoregressive VLM |
| Pretraining | Masked latent prediction | Image-text pairs + instruction tuning |
| Input Frames | 8 | 4 |
| Input Resolution | $224 \times 224$ | Dynamic resolution |
| Feature Dimension | 1,280 | 3,584 |
| Precision | float16 + SDPA | 4-bit NF4 |
| Modality | Video only | Vision + Language |
| Hugging Face ID | facebook/ vjepa2-vith-fpc64-256 | Qwen/Qwen2. 5-VL-7B-Instruct |

## 3.4 Implementation Details

All experiments were implemented as Kaggle Notebooks executing on the cloud platform described in Section 3.3. The complete source code for all pipeline components is provided in Appendix A. This section describes the design rationale, algorithmic strategies, and key implementation decisions for each stage of the experimental pipeline.

### 3.4.1 FineGym Data Pipeline

The FineGym data acquisition pipeline is driven by the official annotation file, which provides YouTube video identifiers paired with precise event-level timestamps (start and end times) for each gymnastic routine. The pipeline operates in three stages:

1. Annotation Parsing: The annotation JSON is parsed to extract YouTube video IDs, event categories (BB, FX, UB, VT), and temporal boundaries for each event segment. A balanced sampling strategy selects 300 clips per apparatus category, yielding 1,200 total clips with uniform class distribution.

2. Temporal Clip Extraction: Each event clip is downloaded from YouTube using `yt-dlp` with the --download-sections flag and --force-keyframes-at-cuts to ensure precise temporal trimming aligned to the annotation timestamps. The worst[ext=mp4] /worst format selector downloads the lowest available quality to minimize bandwidth and storage while preserving sufficient visual information for subsequent frame extraction at $224 \times 224$ resolution (Listing A.1 in Appendix A).

3. Fault-Tolerant Execution: A JSON-based progress log tracks the download status of each clip, enabling the pipeline to resume after interruptions (e.g., Kaggle session timeouts or transient network errors) without re-downloading previously acquired clips. Each download is attempted up to 2 times with a 120-second timeout per attempt.

### 3.4.2 HMDB51 Data Pipeline

The HMDB51 dataset was obtained as a 2.12 GB zip archive from the Hugging Face Hub (`jili5044/hmdb51`) using the `hf_hub_download` API. The standard Hugging Face `datasets` archive was directly downloaded and extracted locally (Listing A.2 in Appendix A).

After extraction, the pipeline constructs a sample index by recursively walking the directory tree. Action class labels are inferred from parent folder names, which follow HMDB51's standard directory-per-class organization. This process successfully loads all 6,754 video clips across 51 action categories with zero indexing errors. The

directory-based approach provides reliable, schema-independent dataset loading that is robust to metadata inconsistencies in third-party repository mirrors.

### 3.4.3 Feature Extraction Pipeline

Feature extraction follows the frozen-feature protocol described in Section 3.2.2, where backbone model weights remain fixed and only downstream classifier parameters are trained. Both V-JEPA 2 and Qwen2.5-VL are loaded sequentially (one at a time) to manage GPU memory constraints within the 15.6 GB VRAM budget of the NVIDIA Tesla T4.

#### V-JEPA 2 Feature Extraction

V-JEPA 2 is loaded in float16 precision with Scaled Dot-Product Attention (SDPA) via the Hugging Face `transformers` library. For each video clip, 8 frames are uniformly sampled and resized to $224 \times 224$ pixels using the official `VJEPA2VideoProcessor`. The processed frames are passed through the frozen encoder under `torch.no_grad()` with mixed-precision autocast to minimize GPU memory consumption. For baseline classifiers, the final-layer hidden states are spatially mean-pooled to produce a single 1,280-dimensional feature vector per clip (Equation 3.15). For HAF-STD Net, hidden states are extracted from 4 intermediate encoder layers and spatially pooled per frame to produce the 4D multi-layer tensor $\mathbf{F}_{\text{multi}} \in \mathbb{R}^{4 \times T' \times 1280}$ (Equation 3.8). The complete extraction procedure is provided in Listing A.3 (Appendix A).

#### Qwen2.5-VL Feature Extraction

After V-JEPA 2 extraction completes, the model is explicitly deleted from GPU memory, CUDA cache is cleared, and Python garbage collection is triggered before loading Qwen2.5-VL. The 7B-parameter model is loaded in 4-bit NF4 quantization (Equation 3.14) with double quantization enabled, reducing the model footprint from approximately 14 GB to approximately 4 GB.

Feature extraction from Qwen2.5-VL requires constructing a chat-template-formatted input for each video clip, as the model is designed for conversational visual question answering rather than direct feature extraction. A minimal prompt ("Describe.") is applied to 4 uniformly sampled frames per clip. The model processes this multimodal input through its Vision Encoder and LM Decoder, and the hidden states from the final decoder layer are mean-pooled across the sequence dimension to produce a 3,584-dimensional feature vector per clip (Equation 3.15). The full quantized extraction procedure is provided in Listing A.4 (Appendix A).

### 3.4.4 Classifier Training Pipeline

#### Classical Machine Learning Classifiers

The three scikit-learn classifiers (SVM-RBF, Random Forest, Logistic Regression) operate on PCA-reduced features and are trained using their default scikit-learn API with the hyperparameters specified in Section 3.2.4. These classifiers require no iterative training loop and are fitted in a single pass on the training split.

#### Neural Network Classifiers

The five neural network classifiers (MLP, DeepMLP, ResMLP, Conv1DNet, MiniTransformer) share a common training harness that implements the regularization techniques described in Section 3.2.5. The training procedure operates as follows:

1. **Optimization:** All models are trained with AdamW optimizer (lr $= 10^{-3}$, weight decay $= 10^{-4}$) and Cosine Annealing learning rate scheduling that decays the learning rate from $10^{-3}$ to $10^{-6}$ over the training horizon.

2. **Feature-Space Mixup:** At each training step, input feature vectors are interpolated with randomly permuted counterparts using a mixing coefficient $\lambda \sim \text{Beta}(0.2, 0.2)$, clamped to $\lambda \geq 0.5$ to ensure the original sample dominates. The cross-entropy loss is computed as a weighted combination over both original and permuted labels.

3. **Label Smoothing:** The one-hot target vectors are softened with $\epsilon = 0.1$, distributing $\frac{\epsilon}{C}$ probability mass to each non-target class to prevent over-confident predictions and improve calibration.

4. **Early Stopping:** Training terminates after 15 consecutive epochs (30 for HAF-STD Net) without improvement in validation accuracy. The model checkpoint with the highest validation accuracy is restored as the final model.

The ResMLP architecture (Listing A.5) and the shared training loop (Listing A.6) are provided in Appendix A.

#### HAF-STD Net Training

The HAF-STD Net training pipeline extends the baseline training harness to accommodate the 4D multi-layer temporal tensor input $\mathbf{F}_{\text{multi}} \in \mathbb{R}^{B\times 4\times T\times 1280}$. Unlike the baseline classifiers that operate on 1D PCA-reduced feature vectors, HAF-STD Net requires the full-dimensional multi-layer representation without PCA reduction, as the spectral-temporal decomposition and depth velocity computation (Section 3.2.4) operate directly on the spatiotemporal structure.

The feature-space mixup augmentation is generalized to 4D tensors by applying element-wise interpolation across all dimensions simultaneously:

$$\tilde{\mathbf{F}} = \lambda \cdot \mathbf{F}_i + (1 - \lambda) \cdot \mathbf{F}_{\pi(i)}, \quad \lambda \sim \text{Beta}(0.2, 0.2) \tag{3.44}$$

where $\pi$ denotes a random permutation of the batch indices. This preserves the multi-layer temporal structure while providing regularization equivalent to the baseline mixup. The complete HAF-STD Net architecture (Listing A.7) and training harness (Listing A.8) are provided in Appendix A.

### 3.4.5 Memory Management and Checkpointing

Given the limited GPU memory (15.6 GB) and the sequential nature of feature extraction from two large models, careful memory management is critical. The implementation employs several strategies:

1. **Sequential Model Loading:** Only one model is loaded at a time. After V-JEPA 2 extraction is complete, the model and processor are explicitly deleted, GPU cache is cleared (`torch.cuda.empty_cache()`), and Python garbage collection is triggered (`gc.collect()`) before loading Qwen2.5-VL.

2. **Periodic Checkpointing:** Features and labels are saved to disk at regular intervals (every 100 clips for FineGym, every 500 videos for HMDB51). If the Kaggle session times out or the kernel restarts, extraction resumes from the last checkpoint rather than restarting.

3. **Periodic Cache Clearing:** `torch.cuda.empty_cache()` is called periodically (every 200–500 videos) to release fragmented GPU memory and prevent out-of-memory errors during long extraction runs.

4. **OOM Error Recovery:** `RuntimeError` exceptions containing "out of memory" are caught, GPU cache is cleared, and extraction continues with the next video rather than terminating the entire pipeline.

### 3.4.6 Data Pipeline Challenges

Several implementation challenges were encountered and resolved:

- **FineGym Dataset Acquisition:** A balanced dataset of 1,200 FineGym clips (300 clips per event category) was acquired using `yt-dlp` with temporal trimming based on annotation timestamps. Approximately 15% of clips required retry attempts due to transient network errors or unavailable videos.

- **HMDB51 Schema Validation Error:** The Hugging Face `datasets` library encountered a `DatasetGenerationError` when loading the HMDB51 dataset through its standard API. This was resolved by bypassing the dataset library entirely and using `hf_hub_download` to directly download the raw zip archive, followed by manual extraction and directory-based indexing.
- **Qwen2.5-VL Memory Constraints:** At full float16 precision, Qwen2.5-VL (7B parameters) requires approximately 14 GB of VRAM, leaving insufficient memory for activations and intermediate computations on the 15.6 GB T4 GPU. The 4-bit NF4 quantization with double quantization reduces the model footprint to approximately 4 GB, providing adequate headroom for inference.

#### 3.4.7 Execution Timeline

**Table 5:** Execution timeline for both dataset experiments.

| Dataset | Stage | Duration |
|---|---|---|
| FineGym | Video Download (`yt-dlp`) | $\sim$110 min |
| | V-JEPA 2 (1,200 clips) | $\sim$12 min |
| | Qwen2.5-VL (1,200 clips) | $\sim$14 min |
| | Classification (9 classifiers) | $<$1 min |
| HMDB51 | Dataset Download (HF Hub) | $\sim$1 min |
| | V-JEPA 2 (6,754 videos) | $\sim$32 min |
| | Qwen2.5-VL (6,754 videos) | $\sim$56 min |
| | Classification (9 classifiers) | $<$5 min |
| **Total** | | **$\sim$210 min** |

Qwen2.5-VL takes approximately $1.75\times$ longer than V-JEPA 2 for feature extraction on HMDB51, despite processing fewer frames per video (4 vs. 8). This is attributable to the larger model size and the overhead of chat template construction and tokenization at each step. The classification stage (all 9 classifiers) adds minimal overhead, with sklearn classifiers completing in seconds and neural network classifiers requiring 1–3 minutes for training with early stopping.

# Chapter 4

# Results and Discussion

## 4.1 Findings and Observations

### 4.1.1 Experiment 1: FineGym (4-Class Fine-Grained Classification)

**Per-Classifier Results**

**Table 6:** Classifier performance on FineGym — V-JEPA 2 features.

| **Classifier** | **Type** | **Accuracy (%)** | **F1 Macro (%)** | **MCC** |
|---|---|---|---|---|
| SVM-RBF | sklearn | 95.0 | 94.8 | 0.933 |
| Random Forest | sklearn | 86.7 | 86.3 | 0.822 |
| LogReg | sklearn | 94.2 | 93.9 | 0.923 |
| MLP | Neural Net | 90.8 | 90.2 | 0.878 |
| DeepMLP | Neural Net | 97.1 | 96.8 | 0.961 |
| ResMLP | Neural Net | 97.9 | 97.5 | 0.972 |
| Conv1DNet | Neural Net | 96.3 | 96.0 | 0.950 |
| Mini Transformer | Neural Net | 97.5 | 97.2 | 0.967 |
| **HAF-STD Net** | **Neural Net** | **98.1** | **97.8** | **0.974** |

**Table 7:** Classifier performance on FineGym — Qwen2.5-VL features.

| Classifier | Type | Accuracy (%) | F1 Macro (%) | MCC |
|---|---|---|---|---|
| SVM-RBF | sklearn | 72.7 | 73.4 | 0.643 |
| Random Forest | sklearn | 58.2 | 59.4 | 0.454 |
| LogReg | sklearn | 83.6 | 82.5 | 0.785 |
| MLP | Neural Net | 63.6 | 62.3 | 0.516 |
| **DeepMLP** | **Neural Net** | **89.1** | **88.4** | **0.855** |
| ResMLP | Neural Net | 83.6 | 82.9 | 0.791 |
| Conv1DNet | Neural Net | 74.5 | 74.4 | 0.664 |
| Mini Transformer | Neural Net | 80.0 | 78.9 | 0.734 |

On FineGym, the neural network classifiers significantly outperform the sklearn classifiers for V-JEPA 2 features. HAF-STD Net achieves the best overall performance (98.1% accuracy, 97.8% F1 macro, MCC = 0.974), surpassing the best baseline classifier ResMLP (97.9% accuracy, 97.5% F1) by exploiting V-JEPA 2's multi-layer hierarchical representations. This indicates that V-JEPA 2's representations are so well-structured that the hierarchical adaptive fusion with spectral-temporal decomposition can near-perfectly separate all four gymnastics event classes. Even a linear probe (LogReg) achieves 94.2%, confirming that V-JEPA 2's self-supervised pretraining produces highly discriminative, near-linearly-separable features for fine-grained action recognition.

For Qwen2.5-VL, DeepMLP (89.1%) outperforms the best sklearn classifier (LogReg, 83.6%) by 5.5 percentage points, demonstrating that the additional non-linear capacity of deep networks can better exploit the structure in Qwen2.5-VL's feature space.

**FineGym Comprehensive Comparison**

**Table 8:** FineGym: Comprehensive metric comparison (best classifiers).

| Metric | V-JEPA 2 | Qwen2.5-VL |
|---|---|---|
| Accuracy (%) | 98.1 | 89.1 |
| Precision macro (%) | 98.5 | 88.9 |
| Precision weighted (%) | 98.1 | 89.1 |
| Recall macro (%) | 98.3 | 88.4 |
| Recall weighted (%) | 98.1 | 89.1 |
| F1 macro (%) | 97.8 | 88.4 |
| F1 weighted (%) | 98.5 | 88.9 |
| MCC | 0.974 | 0.855 |
| Cohen's Kappa | 0.972 | 0.854 |
| Best Classifier | HAF-STD Net | DeepMLP |
| **Winner** | **V-JEPA 2 (+9.4% F1)** | |

V-JEPA 2 with HAF-STD Net achieves near-perfect classification on FineGym with an MCC of 0.974 and Cohen's Kappa of 0.972. The 9.4 percentage point F1 advantage over Qwen2.5-VL represents a substantial difference in representation quality for fine-grained recognition.

### 4.1.2 Experiment 2: HMDB51 (51-Class Coarse-Grained Classification)

**Per-Classifier Results**

**Table 9:** Classifier performance on HMDB51 — V-JEPA 2 features.

| Classifier | Type | Accuracy (%) | F1 Macro (%) | MCC |
|---|---|---|---|---|
| SVM-RBF | sklearn | 79.8 | 81.4 | 0.792 |
| Random Forest | sklearn | 70.6 | 73.4 | 0.706 |
| LogReg | sklearn | 78.2 | 79.4 | 0.776 |
| MLP | Neural Net | 79.3 | 80.3 | 0.788 |
| DeepMLP | Neural Net | 82.9 | 83.5 | 0.824 |
| ResMLP | Neural Net | 82.9 | 83.6 | 0.824 |
| Conv1DNet | Neural Net | 77.8 | 78.5 | 0.772 |
| Mini Transformer | Neural Net | 79.4 | 79.9 | 0.789 |
| **HAF-STD Net** | **Neural Net** | **84.7** | **85.3** | **0.843** |

**Table 10:** Classifier performance on HMDB51 — Qwen2.5-VL features.

| Classifier | Type | Accuracy (%) | F1 Macro (%) | MCC |
|---|---|---|---|---|
| SVM-RBF | sklearn | 73.6 | 75.3 | 0.729 |
| Random Forest | sklearn | 64.2 | 63.8 | 0.635 |
| LogReg | sklearn | 73.6 | 75.3 | 0.729 |
| MLP | Neural Net | 70.8 | 70.4 | 0.702 |
| **DeepMLP** | **Neural Net** | **77.1** | **79.1** | **0.765** |
| ResMLP | Neural Net | 76.7 | 77.6 | 0.760 |
| Conv1DNet | Neural Net | 11.5 | 0.8 | 0.081 |
| Mini Transformer | Neural Net | 71.5 | 72.5 | 0.707 |

On HMDB51, HAF-STD Net achieves the best overall performance with 84.7% accuracy (85.3% F1 macro, MCC = 0.843) for V-JEPA 2, representing a +1.7% improvement over the best baseline classifier (ResMLP, 83.6% F1). Among baseline classifiers, DeepMLP and ResMLP tie at 82.9% accuracy. For Qwen2.5-VL, DeepMLP achieves the best performance at 77.1% accuracy (79.1% F1 macro).

**HMDB51 Comprehensive Comparison**

**Table 11:** HMDB51: Comprehensive metric comparison (best classifiers).

| Metric | V-JEPA 2 | Qwen2.5-VL |
|---|---|---|
| Accuracy (%) | 84.7 | 77.1 |
| Precision macro (%) | 87.7 | 80.3 |
| Precision weighted (%) | 85.9 | 78.3 |
| Recall macro (%) | 84.8 | 79.1 |
| Recall weighted (%) | 84.7 | 77.1 |
| F1 macro (%) | 85.3 | 79.1 |
| F1 weighted (%) | 84.5 | 77.3 |
| MCC | 0.843 | 0.765 |
| Cohen's Kappa | 0.842 | 0.765 |
| Best Classifier | HAF-STD Net | DeepMLP |
| **Winner** | **V-JEPA 2 (+6.2% F1)** | |

On HMDB51, V-JEPA 2 with the HAF-STD Net classifier maintains a substantial advantage across all metrics, with a 6.2 percentage point F1 macro margin and a 7.6 percentage point accuracy margin over Qwen2.5-VL. The HAF-STD Net's improvement

over the best baseline classifier (ResMLP, 83.6% F1) demonstrates that exploiting V-JEPA 2's multi-layer hierarchical representations and temporal structure yields meaningful gains beyond standard feature pooling.

## 4.2 Result Analysis

### 4.2.1 V-JEPA 2 Consistently Outperforms Qwen2.5-VL

Across both datasets, all metrics, and all classifiers, V-JEPA 2 produces more discriminative representations than Qwen2.5-VL. V-JEPA 2 wins on every single metric on both FineGym and HMDB51. On both datasets, HAF-STD Net further widens this gap by exploiting V-JEPA 2's multi-layer depth structure and temporal coherence, achieving a 9.4% F1 advantage on FineGym and a 6.2% F1 advantage on HMDB51 over Qwen2.5-VL.

### 4.2.2 Performance Gap Depends on Task Granularity

V-JEPA 2's advantage is substantially larger on the fine-grained FineGym task (+9.4% F1 with HAF-STD Net) than on the coarse-grained HMDB51 task (+6.2% F1 with HAF-STD Net).

- V-JEPA 2's self-supervised training on masked spatio-temporal prediction develops representations well-suited for discriminating subtle temporal patterns within a narrow visual domain.
- Qwen2.5-VL's generative training produces general-purpose representations encoding broad descriptive information (object identity, scene context, textual semantics).

### 4.2.3 Parameter Efficiency

V-JEPA 2 (654M) consistently outperforms Qwen2.5-VL (7B) despite having $\sim 10\times$ fewer parameters. On HMDB51, V-JEPA 2 with HAF-STD Net achieves 84.7% accuracy vs. Qwen2.5-VL's 77.1%, delivering 7.6% higher accuracy with $10\times$ fewer parameters. The HAF-STD Net head itself adds only 4.8M parameters (0.7% of V-JEPA 2's backbone), demonstrating that intelligent probe design can yield substantial gains at minimal parameter cost.

# Chapter 5

# Conclusion and Recommendations

## 5.1 Conclusions

This thesis presented a comprehensive comparative analysis of two fundamentally different video representation learning paradigms—self-supervised learning and vision-language modelling—evaluated across two complementary benchmark datasets. V-JEPA 2 (654M parameters), a self-supervised video encoder, was compared against Qwen2.5-VL (7B parameters), a vision-language model, using a controlled frozen-feature evaluation protocol with nine downstream classifiers, including the proposed HAF-STD Net.

The experimental results demonstrate that self-supervised video encoders produce superior temporal representations for video action recognition. V-JEPA 2 outperformed Qwen2.5-VL on both datasets—achieving 97.8% vs. 88.4% macro F1 (98.1% vs. 89.1% accuracy, MCC = 0.974 vs. 0.855) on FineGym with HAF-STD Net/DeepMLP, and 85.3% vs. 79.1% macro F1 (84.7% vs. 77.1% accuracy, MCC = 0.843 vs. 0.765) on HMDB51 with HAF-STD Net/DeepMLP. This advantage is most pronounced on fine-grained tasks: the 9.4 percentage point F1 gap on FineGym narrows to 6.2 percentage points on HMDB51, indicating that the JEPA paradigm's masked spatio-temporal prediction is especially valuable for discriminating subtle temporal motion patterns within a narrow visual domain.

Furthermore, the results confirm that task specialization outweighs model scale. V-JEPA 2 achieves superior performance with roughly 10$\times$ fewer parameters (654M vs 7B), demonstrating that a video-native self-supervised architecture is more effective than a larger general-purpose VLM for action recognition.

The proposed HAF-STD Net achieves the best performance on both benchmarks. The Hierarchical Adaptive Fusion with Spectral-Temporal Decomposition Network outperforms all eight baseline classifiers on both FineGym (98.1% accuracy, 97.8% F1, MCC = 0.974) and HMDB51 (84.7% accuracy, 85.3% F1, MCC = 0.843), demonstrating that exploiting multi-layer depth representations through wavelet frequency decomposition and adaptive routing yields consistent gains over standard

feature pooling approaches.

## 5.2 Recommendations for Further Work

Based on the findings of this research, several directions are recommended for future investigation. The evaluation should be extended to FineGym's set-level (288 classes) and element-level (530 classes) tasks for deeper granularity analysis of fine-grained sub-action classification. The frozen-feature study should be complemented with end-to-end fine-tuning to assess whether VLM features benefit more from task-specific adaptation than SSL features. Additionally, evaluating Qwen2.5-VL at full precision (float16) on higher-end GPUs would help eliminate the potential confound of 4-bit quantization effects on representation quality.

Future work should also investigate fusing V-JEPA 2 temporal representations with Qwen2.5-VL semantic representations through cross-attention mechanisms to explore whether complementary paradigm strengths can be combined for improved performance. Finally, benchmarking on specialized real-world domains such as surgical video analysis, physiotherapy monitoring, and fine-grained athletic motion analysis would establish the practical applicability of these findings beyond standard academic benchmarks.

# References


Assran, M., Bardes, A., Fan, D., Garrido, Q., Howes, R., Komeili, M., Muckley, M., Rizvi, A., Roberts, C., Sinha, K., Zholus, A., Arnaud, S., Gejji, A., Martin, A., Hogan, F. R., Dugas, D., Bojanowski, P., Khalidov, V., Labatut, P., … Ballas, N. (2025). V-JEPA 2: Self-supervised video models enable understanding, prediction and planning.

Assran, M., Duval, Q., Misra, I., Bojanowski, P., Vincent, P., Rabbat, M., LeCun, Y., & Ballas, N. (2023). Self-supervised learning from images with a joint-embedding predictive architecture. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 15619–15629.

Bagatella, M., Pirotta, M., Touati, A., Lazaric, A., & Tirinzoni, A. (2025). TD-JEPA: Latent-predictive representations for zero-shot reinforcement learning.

Bai, S., Yang, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Li, C., Liu, C., Huang, C., et al. (2025). Qwen2.5-VL technical report.

Bardes, A., Garrido, Q., Ponce, J., Chen, X., Rabbat, M., LeCun, Y., Assran, M., & Ballas, N. (2024). Revisiting feature prediction for learning visual representations from video. *Proceedings of the International Conference on Machine Learning (ICML)*.

Bordes, F., Pang, R. Y., Ajay, A., Li, A. C., Bardes, A., Petryk, S., Mañas, O., Lin, Z., Mahmoud, A., Jayaraman, B., Ibrahim, M., Hall, M., Xiong, Y., Lebensold, J., Ross, C., Jayakumar, S., Guo, C., Bouchacourt, D., Al-Tahan, H., & Chandra, V. (2024). An introduction to vision-language modeling.

Breiman, L. (2001). Random forests. *Machine Learning*, *45*(1), 5–32.

Chen, D., Shukor, M., Moutakanni, T., Chung, W., Yu, J., Kasarla, T., Bang, Y., Bolourchi, A., LeCun, Y., & Fung, P. (2026). VL-JEPA: Joint embedding predictive architecture for vision-language.

Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, *20*(3), 273–297.

Cox, D. R. (1958). The regression analysis of binary sequences. *Journal of the Royal Statistical Society: Series B (Methodological)*, *20*(2), 215–242.

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778.

Huang, H., LeCun, Y., & Balestriero, R. (2025). LLM-JEPA: Large language models meet joint embedding predictive architectures.

Huang, Y. (2026). VJEPA: Variational joint embedding predictive architectures as probabilistic world models.

Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, 448–456.

Kuehne, H., Jhuang, H., Garrote, E., Poggio, T., & Serre, T. (2011). HMDB: A large video database for human motion recognition. *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2556–2563.

LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, *86*(11), 2278–2324.

Lin, L., Jain, A., Liu, Y., & Fragkiadaki, K. (2026). Qwen-3D: A generalist 3D vision-language model for spatial understanding.

Liu, B., Chen, Z., & Woodruff, D. P. (2026). JEPA-Reasoner: Decoupling latent reasoning from token generation.

Mur-Labadia, L., Muckley, M., Bar, A., Assran, M., Sinha, K., Rabbat, M., LeCun, Y., Ballas, N., & Bardes, A. (2026). V-JEPA 2.1: Unlocking dense features in video self-supervised learning.

Pathak, S., & Han, B. (2026). Towards efficient large vision-language models: A comprehensive survey on inference strategies.

Qwen Team. (2025). Qwen3-VL technical report.

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, *323*(6088), 533–536.

Shao, D., Zhao, Y., Dai, B., & Lin, D. (2020). FineGym: A hierarchical video dataset for fine-grained action understanding. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2616–2625.

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, *15*(56), 1929–1958.

Terver, B., & Yang, T.-Y. (2026). What drives success in physical planning with joint-embedding predictive world models?

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems (NeurIPS)*, *30*, 5998–6008.

Zhang, H., Li, Y., He, S., Nagarajan, T., Chen, M., Lu, J., Li, A., & Fu, Y. (2026). ThinkJEPA: Empowering latent world models with large vision-language reasoning model.

Zhang, Y., et al. (2023). Vision-language models for vision tasks: A survey.

# Appendix A

# Source Code Listings

This appendix provides the complete Python source code for all major components of the experimental pipeline described in Chapter 3. All experiments were implemented in Kaggle Notebooks using Python 3.12 and PyTorch.

## A.1 Data Acquisition

**Listing A.1:** FineGym clip download with `yt-dlp` temporal trimming.

```
def download_clip(yt_id, start, end, out_path, max_retries=2):
    """Download a clip from YouTube using yt-dlp."""
    if os.path.exists(out_path) and \
       os.path.getsize(out_path) > 1000:
        return True # Already downloaded
    url = f'https://www.youtube.com/watch?v={yt_id}'
    for attempt in range(max_retries):
        try:
            cmd = [
                'yt-dlp', '--quiet', '--no-warnings',
                '--download-sections', f'*{start}-{end}',
                '-f', 'worst[ext=mp4]/worst',
                '--force-keyframes-at-cuts',
                '-o', out_path, url
            ]
            subprocess.run(
                cmd, capture_output=True, text=True, timeout=120)
            if os.path.exists(out_path) and \
               os.path.getsize(out_path) > 500:
                return True
        except (subprocess.TimeoutExpired, Exception):
            pass
    return False
```

**Listing A.2:** HMDB51 download and local indexing pipeline.

```
from huggingface_hub import hf_hub_download

# Download raw zip directly (bypasses schema errors)
archive_path = hf_hub_download(
    repo_id="jili5044/hmdb51",
    filename="hmdb51.zip",
    repo_type="dataset",
```

```
    local_dir=download_dir
)

# Extract and index videos by class folder
with zipfile.ZipFile(archive_path, 'r') as zip_ref:
    zip_ref.extractall(extract_dir)

# Build sample list from directory structure
ds = []
for root, dirs, files in os.walk(extract_dir):
    for file in files:
        if file.endswith(('.mp4', '.avi', '.mkv')):
            video_path = os.path.join(root, file)
            label_name = os.path.basename(root)
            ds.append({
                'video_path': video_path,
                'label': label_name
            })
```

## A.2 Feature Extraction

**Listing A.3:** V-JEPA 2 feature extraction with checkpointing.

```
from transformers import VJEPA2Model, VJEPA2VideoProcessor

vproc = VJEPA2VideoProcessor.from_pretrained(
    'facebook/vjepa2-vith-fpc64-256')
vmodel = VJEPA2Model.from_pretrained(
    'facebook/vjepa2-vith-fpc64-256',
    torch_dtype=torch.float16,
    attn_implementation='sdpa',
).to(device).eval()

# Feature extraction with checkpointing
for i in range(start_idx, total):
    frames = load_frames(clip['path'], 8, 224)
    inputs = vproc(frames, return_tensors='pt')
    inputs = {k: v.to(device) for k, v in inputs.items()}
    with torch.no_grad():
        with torch.amp.autocast('cuda'):
            out = vmodel(**inputs)
            feat = (out.last_hidden_state.mean(dim=1)
                    .cpu().float().squeeze(0))
    features.append(feat)
```

**Listing A.4:** Qwen2.5-VL 4-bit quantized feature extraction.

```
from transformers import (
    Qwen2_5_VLForConditionalGeneration,
    AutoProcessor, BitsAndBytesConfig)

bnb = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_quant_type='nf4',
    bnb_4bit_use_double_quant=True)
```

```
qmodel = Qwen2_5_VLForConditionalGeneration.from_pretrained(
   'Qwen/Qwen2.5-VL-7B-Instruct',
   quantization_config=bnb,
   device_map='auto',
   torch_dtype=torch.float16)
qproc = AutoProcessor.from_pretrained(
   'Qwen/Qwen2.5-VL-7B-Instruct')

# Chat template construction for feature extraction
messages = [{'role': 'user', 'content': [
   {'type': 'video', 'video': frames},
   {'type': 'text', 'text': 'Describe.'},
]}]
text = qproc.apply_chat_template(
   messages, tokenize=False, add_generation_prompt=True)
inputs = qproc(text=[text], videos=[frames],
           return_tensors='pt', padding=True)

with torch.no_grad():
   out = qmodel(**inputs, output_hidden_states=True)
   feat = (out.hidden_states[-1].mean(dim=1)
         .cpu().float().squeeze(0))
```

## A.3 Classifier Architectures

**Listing A.5:** ResMLP architecture with residual blocks.

```
class ResBlock(nn.Module):
   def __init__(self, dim, dropout=0.2):
      super().__init__()
      self.net = nn.Sequential(
         nn.Linear(dim, dim),
         nn.BatchNorm1d(dim), nn.GELU(),
         nn.Dropout(dropout),
         nn.Linear(dim, dim),
         nn.BatchNorm1d(dim),
      )
      self.act = nn.GELU()

   def forward(self, x):
      return self.act(x + self.net(x))

class ResMLP(nn.Module):
   def __init__(self, in_dim, num_classes, hidden=256, n_blocks=3):
      super().__init__()
      self.proj = nn.Sequential(
         nn.Linear(in_dim, hidden),
         nn.BatchNorm1d(hidden), nn.GELU())
      self.blocks = nn.Sequential(
         *[ResBlock(hidden) for _ in range(n_blocks)])
      self.head = nn.Linear(hidden, num_classes)

   def forward(self, x):
      return self.head(self.blocks(self.proj(x)))
```

**Listing A.6:** Training loop with label smoothing, mixup augmentation, and cosine LR scheduling.

```
LABEL_SMOOTHING = 0.1

def mixup_data(x, y, alpha=0.2):
    """Feature-space mixup augmentation."""
    lam = np.random.beta(alpha, alpha)
    lam = max(lam, 1 - lam)
    idx = torch.randperm(x.size(0))
    return lam * x + (1-lam) * x[idx], y, y[idx], lam

def train_model(model, X_tr, y_tr, X_ev, y_ev,
                epochs=300, lr=1e-3, patience=30):
    opt = torch.optim.AdamW(
        model.parameters(), lr=lr, weight_decay=1e-4)
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
        opt, T_max=epochs)
    best_acc, best_state, wait = 0, None, 0

    for epoch in range(epochs):
        model.train()
        for xb, yb in dataloader:
            xb_mix, ya, yb_mix, lam = mixup_data(xb, yb)
            logits = model(xb_mix)
            loss_a = F.cross_entropy(
                logits, ya, label_smoothing=LABEL_SMOOTHING)
            loss_b = F.cross_entropy(
                logits, yb_mix, label_smoothing=LABEL_SMOOTHING)
            loss = lam * loss_a + (1 - lam) * loss_b
            opt.zero_grad()
            loss.backward()
            opt.step()
        scheduler.step()

        # Early stopping on eval accuracy
        model.eval()
        with torch.no_grad():
            preds = model(X_ev).argmax(1)
            acc = (preds == y_ev).float().mean().item()
        if acc > best_acc:
            best_acc = acc
            best_state = model.state_dict()
            wait = 0
        else:
            wait += 1
            if wait >= patience: break
    model.load_state_dict(best_state)
    return model
```

## A.4 HAF-STD Net Implementation

**Listing A.7:** HAF-STD Net architecture and spectral-temporal decomposition (HMDB51 configuration).

```
class HaarDWT1D(nn.Module):
```

```
    """1D Haar Discrete Wavelet Transform along temporal axis.
    Decomposes temporal signal into low-frequency (appearance/trend)
    and high-frequency (motion/change) components.
    """
    def forward(self, x):
        """x: (B, T, D) -> (low: (B, T//2, D), high: (B, T//2, D))"""
        if x.shape[1] % 2 != 0:
            x = F.pad(x, (0, 0, 0, 1)) # pad temporal dim if odd
        x_even = x[:, 0::2, :] # Even-indexed temporal positions
        x_odd = x[:, 1::2, :] # Odd-indexed temporal positions
        # Approximation = appearance/trend
        low = (x_even + x_odd) * 0.5
        # Detail = motion/change dynamics
        high = (x_even - x_odd) * 0.5
        return low, high


class HAFSTD_Net(nn.Module):
    """Hierarchical Adaptive Fusion with Spectral-Temporal Decomposition.
    Novel architecture for V-JEPA 2 on HMDB51 (51 classes).
    """
    def __init__(self, embed_dim=1280, num_layers=4, num_classes=51,
                 hidden=256, dropout=0.3):
        super().__init__()
        self.num_layers = num_layers
        # 3 streams: appearance, motion, velocity
        self.num_streams = num_layers * 3
        self.dwt = HaarDWT1D()

        # Dedicated projection heads (12 total)
        self.stream_projections = nn.ModuleList([
            nn.Sequential(
                nn.Linear(embed_dim, hidden),
                nn.LayerNorm(hidden),
                nn.GELU(),
                nn.Dropout(dropout),
            )
            for _ in range(self.num_streams)
        ])

        # Adaptive routing network over 12 depth x frequency streams
        self.router = nn.Sequential(
            nn.Linear(self.num_streams * hidden, hidden),
            nn.GELU(),
            nn.Dropout(dropout),
            nn.Linear(hidden, self.num_streams),
        )

        # Classification head with cosine similarity
        self.final_proj = nn.Sequential(
            nn.Linear(hidden, hidden),
            nn.LayerNorm(hidden),
            nn.GELU(),
            nn.Dropout(dropout),
        )
        self.class_weights = nn.Parameter(
            torch.randn(num_classes, hidden))
        self.scale = nn.Parameter(torch.tensor(16.0))

```

```
    def forward(self, x):
        """x: (B, num_layers=4, T, D=1280)"""
        B = x.shape[0]
        streams = []

        for k in range(self.num_layers):
            layer_feat = x[:, k, :, :] # (B, T, D)

            # 1. Spectral-Temporal Decomposition via Haar DWT
            low, high = self.dwt(layer_feat)
            appearance = low.mean(dim=1) # low-freq = appearance
            motion = high.mean(dim=1) # high-freq = motion dynamics

            # 2. Depth Representation Velocity
            if k < self.num_layers - 1:
                next_layer = x[:, k + 1, :, :].mean(dim=1)
                curr_layer = layer_feat.mean(dim=1)
                velocity = next_layer - curr_layer
            else:
                velocity = torch.zeros_like(appearance)

            # Project each stream through its dedicated head
            idx = k * 3
            streams.append(self.stream_projections[idx](appearance))
            streams.append(self.stream_projections[idx + 1](motion))
            streams.append(self.stream_projections[idx + 2](velocity))

        # Stack all 12 streams: (B, 12, hidden)
        all_streams = torch.stack(streams, dim=1)

        # 3. Adaptive Routing
        flat = all_streams.reshape(B, -1)
        routing_weights = torch.softmax(self.router(flat), dim=-1)
        fused = (all_streams * routing_weights.unsqueeze(-1)).sum(dim=1)

        # 4. Cosine Similarity Classification
        fused = self.final_proj(fused)
        fused_norm = F.normalize(fused, p=2, dim=-1)
        weight_norm = F.normalize(self.class_weights, p=2, dim=-1)
        logits = self.scale * fused_norm @ weight_norm.T
        return logits
```

**Listing A.8:** 4D Tensor Mixup and Training Harness for HAF-STD Net on HMDB51.

```
def train_hafstd_model(model, X_tr_4d, y_tr, X_ev_4d, y_ev,
                       epochs=300, lr=1e-3, batch_size=64,
                       patience=30, use_mixup=True):
    """Train HAF-STD Net on 4D per-frame features (N, layers, T, D)."""
    model = model.to(device)
    opt = torch.optim.AdamW(
        model.parameters(), lr=lr, weight_decay=1e-4)
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
        opt, T_max=epochs)

    Xt = torch.tensor(X_tr_4d, dtype=torch.float32)
    yt = torch.tensor(y_tr, dtype=torch.long)
    Xe = torch.tensor(X_ev_4d, dtype=torch.float32).to(device)
    ye = torch.tensor(y_ev, dtype=torch.long).to(device)

```

```
    dl = DataLoader(
        TensorDataset(Xt, yt), batch_size=batch_size, shuffle=True)
    best_acc, best_state, wait = 0, None, 0

    for epoch in range(epochs):
        model.train()
        for xb, yb in dl:
            xb, yb = xb.to(device), yb.to(device)
            if use_mixup and xb.size(0) > 1:
                lam = max(np.random.beta(0.2, 0.2), 0.5)
                idx = torch.randperm(xb.size(0))
                xb_mix = lam * xb + (1 - lam) * xb[idx]
                logits = model(xb_mix)
                loss_a = F.cross_entropy(
                    logits, yb, label_smoothing=LABEL_SMOOTHING)
                loss_b = F.cross_entropy(
                    logits, yb[idx], label_smoothing=LABEL_SMOOTHING)
                loss = lam * loss_a + (1 - lam) * loss_b
            else:
                loss = F.cross_entropy(
                    model(xb), yb, label_smoothing=LABEL_SMOOTHING)
            opt.zero_grad(); loss.backward(); opt.step()
        scheduler.step()

        model.eval()
        with torch.no_grad():
            acc = (model(Xe).argmax(1) == ye).float().mean().item()
        if acc > best_acc:
            best_acc = acc
            best_state = {
                k: v.cpu().clone()
                for k, v in model.state_dict().items()
            }
            wait = 0
        else:
            wait += 1
            if wait >= patience: break
    model.load_state_dict(
        {k: v.to(device) for k, v in best_state.items()})
    return model
```